

Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores

(Computer Engineering School)

Programa de Licenciatura en Ingeniería en Computadores

(Licentiate Degree Program in Computer Engineering)



**Modulación y Demodulación de Señales en Sistemas
Embebidos - ModuTEC**

(Signal Modulation and Demodulation in Embedded Systems - ModuTEC)

**Informe de Trabajo de Graduación para optar por el título de Ingeniero en
Computadores con grado académico de Licenciatura**

(Report of Graduation Work in fulfillment of the requirements for the degree of Licentiate in Computer Engineering)

Mauricio Antonio Calderón Chavarría

Cartago, 17 de mayo de 2026

(Cartago, May 17, 2026)

a

Agradecimientos

El resultado de este trabajo no hubiese sido posible sin el apoyo de

Mauricio Antonio Calderón Chavarría

Cartago, 17 de mayo de 2026

Resumen

Palabras clave:

Abstract

Keywords:

Índice general

Índice de figuras	III
Índice de tablas	IV
1. Introducción	1
1.1. Antecedentes del proyecto	2
1.1.1. Descripción de la organización	2
1.1.2. Descripción del área de conocimiento del proyecto	2
1.1.3. Trabajos similares encontrados	2
1.2. Planteamiento del problema	3
1.2.1. Contexto del problema	3
1.2.2. Justificación del problema	4
1.2.3. Enunciado del problema	5
1.3. Objetivos del proyecto	5
1.3.1. Objetivo general	5
1.3.2. Objetivos específicos	5
1.4. Alcances, entregables y limitaciones del proyecto	6
1.4.1. Alcances	6
1.4.2. Entregables	6
1.4.3. Limitaciones	6
2. Marco teórico	8
2.1. Mapa conceptual del marco teórico	8
2.2. Fundamentos de procesamiento de señales	9
2.2.1. Definición de señal	9
2.2.2. Señales en el dominio del tiempo y la frecuencia	11
2.2.3. Señales analógicas y digitales	12
2.2.4. Muestreo y discretización	12
2.2.5. Representación espectral	14
2.3. Modulación de señales	14
2.3.1. Definición de modulación	14
2.3.2. Modulación analógica	16
2.3.3. Modulación digital	18
2.4. Demodulación de señales	19

2.4.1.	Definición de demodulación	19
2.4.2.	Demodulación de AM	20
2.4.3.	Demodulación de señales digitales	22
2.5.	Sistemas embebidos para procesamiento de señales	23
2.5.1.	Definición de sistema embebido	23
2.5.2.	Restricciones de hardware	23
2.5.3.	Procesamiento en tiempo real	24
3.	Marco metodológico	25
3.1.	Estrategia general de solución	25
3.1.1.	Enfoque metodológico adoptado	25
3.1.2.	Enfoque técnico adoptado	26
3.1.3.	Justificación del enfoque	27
3.2.	Estrategia de implementación	28
3.2.1.	Descripción de las etapas del proceso	29
3.3.	Estrategia de validación y verificación	30
3.4.	Herramientas y tecnologías consideradas	31
3.5.	Desglose del marco metodológico	31
4.	Descripción del trabajo realizado	33
4.1.	Descripción de la arquitectura de la solución	33
4.2.	Descripción del proceso de solución	34
4.2.1.	Adaptación de los algoritmos al entorno embebido	34
4.2.2.	Desarrollo del sistema de visualización y monitoreo	45
4.2.3.	Optimización del procesamiento y flujo de datos en el sistema embebido	50
4.2.4.	Implementación del proceso de evaluación y análisis de desempeño	55
4.3.	Análisis de los resultados obtenidos	55
4.3.1.	Resultados asociados a la integración de los algoritmos de modulación y demodulación	55
4.3.2.	Resultados asociados al sistema de visualización y análisis de señales	55
4.3.3.	Resultados asociados a la optimización del procesamiento y flujo de datos	55
4.3.4.	Resultados asociados al proceso de evaluación y análisis de desempeño	55
5.	Conclusiones y Recomendaciones	56
6.	Apéndices y Anexos	57
	Bibliografía	58

Índice de figuras

2.1. Mapa conceptual del marco teórico. Fuente: Elaboración propia.	9
2.2. Parámetros fundamentales de una señal senoidal. Fuente: Elaboración Propia.	10
2.3. Representación de una señal compuesta en el dominio del tiempo y su correspondiente espectro de magnitud. Fuente: Elaboración Propia.	11
2.4. Ejemplo de muestreo de una señal continua. Fuente: Elaboración Propia.	13
2.5. Señal modulada en amplitud (AM) en el dominio del tiempo y su representación en el dominio de la frecuencia. Fuente: Elaboración Propia.	17
2.6. Modulación digital por amplitud tipo OOK en el dominio del tiempo. Fuente: Elaboración Propia.	19
2.7. Proceso de filtrado en demodulación coherente. Fuente: Elaboración propia.	21
3.1. Representación del enfoque iterativo-incremental. Fuente: Adaptado de Pressman [18].	26
3.2. Proceso general de co-diseño hardware/software. Fuente: Elaboración propia.	27
3.3. Proceso metodológico para el desarrollo del sistema. Fuente: Elaboración propia.	29
4.1. Diagrama de arquitectura de la solución. Fuente: Elaboración propia.	34
4.2. Esquemático general del módulo físico de control utilizado por el sistema embebido.	37
4.3. Estructura general propuesta para el frame serial sincronizado.	48

Índice de tablas

3.1. Desglose del marco metodológico por objetivos.	32
4.1. Comparación entre plataformas embebidas consideradas.	35
4.2. Características técnicas relevantes del RP2040.	36
4.3. Asignación de controles analógicos por técnica implementada.	44
4.4. Comparación entre Matplotlib y PyQtGraph para visualización continua .	47
4.5. Estructura del frame serial transmitido por el sistema embebido.	52
4.6. Interfaz genérica establecida para los módulos de técnica en el sistema embebido.	53
4.7. Parámetros definidos en el archivo de configuración <code>config.json</code>	54

Capítulo 1

Introducción

La modulación y demodulación de señales constituye la base de los sistemas modernos de comunicación, permitiendo la transmisión eficiente de información a través de distintos medios físicos [1]. Su estudio suele apoyarse en simuladores o equipos especializados que permiten observar el comportamiento de las señales bajo condiciones controladas [2]. Sin embargo, estas herramientas no siempre están disponibles en entornos educativos o de prototipado de bajo costo, lo cual limita la experimentación práctica con técnicas tanto analógicas como digitales.

Esta limitación adquiere mayor relevancia al considerar que los entornos de simulación no reflejan completamente las restricciones inherentes a los sistemas embebidos. Aspectos como la capacidad de procesamiento, la memoria disponible y las latencias de ejecución influyen directamente en el comportamiento de los algoritmos de procesamiento de señales, y su impacto no puede ser evaluado de manera adecuada en plataformas puramente simuladas [3].

En este contexto, surge la necesidad de trasladar la ejecución de dichas técnicas hacia entornos de hardware con recursos limitados, de manera que sea posible analizar su comportamiento bajo condiciones operativas concretas. Este proyecto propone el desarrollo de un sistema embebido capaz de ejecutar en tiempo real técnicas de modulación y demodulación, específicamente AM y ASK/OOK, mediante la adaptación de algoritmos de procesamiento de señales a un microcontrolador con restricciones de cómputo.

La solución desarrollada integra la generación de señales, su procesamiento dentro del dispositivo embebido y la transmisión de datos hacia una aplicación externa en un computador personal para su visualización y análisis. Esto permite no solo observar el comportamiento de las señales moduladas y demoduladas, sino también evaluar el desempeño de los algoritmos en función de las limitaciones propias del hardware.

De esta forma, el proyecto no se limita a reproducir el comportamiento teórico de estas técnicas, sino que analiza su desempeño en condiciones reales de ejecución, aportando evidencia práctica sobre la implementación de algoritmos de procesamiento de señales en plataformas embebidas de bajo costo, y contribuyendo a la comprensión de los compromisos entre complejidad algorítmica y restricciones de hardware.

1.1. Antecedentes del proyecto

1.1.1. Descripción de la organización

El proyecto se desarrollará en el Instituto Tecnológico de Costa Rica (TEC), una institución pública de educación superior fundada en 1971, reconocida por su enfoque en la formación científica, tecnológica e investigativa [4]. Su sede principal se ubica en Cartago, y cuenta con diversas unidades académicas distribuidas en todo el país, orientadas al desarrollo de conocimiento y la generación de soluciones tecnológicas aplicadas.

Específicamente, el proyecto se llevará a cabo en la Escuela de Ingeniería en Computadores, unidad académica que forma profesionales en áreas como sistemas embebidos, arquitectura de computadoras y desarrollo de software. La Escuela proporciona laboratorios especializados, asesoría técnica y un ambiente propicio para el desarrollo de proyectos de aplicación tecnológica orientados a resolver necesidades del entorno nacional.

1.1.2. Descripción del área de conocimiento del proyecto

El proyecto se enmarca en el área de desarrollo de algoritmos en sistemas empujados, al abordar la implementación de técnicas de modulación y demodulación dentro de un microcontrolador con recursos limitados. Esto involucra la interacción entre el procesamiento de señales y la ejecución de algoritmos en plataformas de hardware específicas.

Estas técnicas forman parte del procesamiento de señales aplicado a sistemas de comunicación [2], donde el análisis y manipulación de señales es fundamental para la transmisión y recuperación de información. En este contexto, el proyecto integra conceptos asociados tanto al tratamiento de señales como a su implementación en sistemas computacionales.

Desde la perspectiva de los sistemas embebidos, la ejecución de estos algoritmos requiere considerar restricciones propias del hardware, como la capacidad de procesamiento, la memoria disponible y la operación en tiempo real [3]. Estas condiciones influyen directamente en la forma en que los algoritmos deben ser adaptados para su ejecución en un entorno de hardware real.

1.1.3. Trabajos similares encontrados

Diversos trabajos han abordado la implementación de técnicas de modulación y demodulación en sistemas embebidos, evidenciando diferentes enfoques según la capacidad del hardware y el nivel de complejidad requerido.

Su et al. desarrollaron un sistema de comunicación acústica submarina basado en un microcontrolador STM32, en el cual se integran procesos de modulación, demodulación y sincronización, junto con mecanismos de procesamiento de señales en tiempo real [5]. Este trabajo demuestra la viabilidad de implementar sistemas completos de comunicación

en plataformas embebidas, incorporando múltiples etapas del procesamiento dentro del mismo dispositivo. Este enfoque resulta particularmente valioso al evidenciar cómo estructurar una arquitectura embebida capaz de ejecutar de forma integrada las diferentes etapas del procesamiento de señales.

De manera complementaria, Yan et al. propusieron un sistema embebido integrado para comunicación y detección submarina, en el cual se emplean técnicas avanzadas de modulación como OFDM, junto con estimación de canal y procesamiento conjunto de señales [6]. Tanto este trabajo como el [5] abordan sistemas de comunicación completos en entornos embebidos, sin embargo, el uso de técnicas más complejas en [6], implica un incremento significativo en los requerimientos computacionales. Esta comparación permite identificar cómo la selección de la técnica de modulación impacta directamente en la complejidad del sistema, aspecto clave al momento de definir soluciones viables en hardware limitado.

Por otro lado, Xie et al. desarrollaron un sistema de modulación y demodulación AM implementado sobre una arquitectura híbrida basada en STM32 y ZYNQ, incorporando generación de señales mediante DDS, modulación analógica y demodulación en hardware [7]. Este trabajo aporta una referencia directa sobre la implementación práctica de modulación AM en un entorno embebido, técnica que constituye uno de los ejes centrales del presente proyecto. La forma en que se integran los distintos bloques funcionales para generar, modular y recuperar la señal resulta especialmente relevante para comprender las consideraciones necesarias en la implementación de esta técnica en hardware embebido.

En contraste, Morales-Aragones et al. desarrollaron un sistema de comunicación sobre líneas de potencia utilizando microcontroladores de bajo costo, en el cual se emplea modulación ASK como mecanismo principal de transmisión [8]. Este enfoque destaca por su simplicidad y eficiencia en el uso de recursos, demostrando la viabilidad de implementar sistemas funcionales en hardware altamente restringido. La utilización de ASK, técnica también contemplada en este proyecto, permite analizar cómo adaptar algoritmos de modulación a condiciones de hardware limitadas, manteniendo funcionalidad operativa.

A partir de estos trabajos, se observa que las soluciones existentes cubren un amplio espectro, desde sistemas de comunicación complejos con técnicas avanzadas y altos requerimientos de cómputo, hasta implementaciones simplificadas orientadas a hardware de bajo costo. Asimismo, se identifican aportes específicos relevantes, como la integración de sistemas completos de comunicación, la implementación de técnicas particulares como AM y ASK, y la adaptación de algoritmos a diferentes niveles de restricción computacional.

1.2. Planteamiento del problema

1.2.1. Contexto del problema

La modulación y demodulación de señales constituye un elemento fundamental en los sistemas modernos de comunicación, siendo utilizada en tecnologías como radio, transmisión

digital, enlaces inalámbricos y sistemas electrónicos de control [1]. No obstante, el estudio práctico de estas técnicas suele depender de simuladores especializados o equipos de laboratorio que permiten observar el comportamiento de las señales moduladas, recursos que no siempre están disponibles en entornos educativos o de prototipado de bajo costo [2].

En el ámbito de los sistemas embebidos, la implementación de procesos de modulación y demodulación en tiempo real presenta desafíos asociados a las restricciones de cómputo, memoria y sincronización propias de este tipo de dispositivos [3]. Estas limitaciones obligan a adaptar y optimizar los algoritmos de procesamiento de señales para garantizar su ejecución bajo condiciones reales, lo cual no siempre es considerado en entornos de simulación tradicionales.

Como consecuencia, la experimentación directa con técnicas analógicas y digitales en plataformas embebidas accesibles se ve condicionada por la necesidad de equilibrar la complejidad algorítmica con los recursos disponibles. Esto dificulta la validación práctica de los principios de comunicación en condiciones reales de ejecución. Adicionalmente, gran parte de las herramientas de análisis operan en computadores personales, sin evaluar el comportamiento del procesamiento dentro del hardware embebido, lo que limita la comprensión integral del desempeño de estas técnicas en sistemas reales.

1.2.2. Justificación del problema

El desarrollo de un sistema embebido capaz de ejecutar técnicas de modulación y demodulación de forma directa responde a una necesidad creciente en el ámbito de la formación y experimentación en comunicaciones. Actualmente, gran parte de los procesos de análisis de señales dependen de simuladores de software o equipos especializados [2], los cuales no reflejan completamente las restricciones reales del hardware ni permiten evaluar el desempeño de los algoritmos en condiciones de operación limitadas [3].

Adicionalmente, estas herramientas suelen requerir plataformas de mayor capacidad computacional, lo que dificulta su utilización en entornos educativos o en proyectos de bajo costo. En este contexto, la implementación de técnicas de modulación y demodulación en un microcontrolador accesible permite trasladar estos procesos a un entorno de ejecución real, habilitando el análisis directo de señales dentro de un sistema físico.

Esto representa un aporte relevante al posibilitar la experimentación práctica sin depender de infraestructura externa compleja. Asimismo, la adaptación de algoritmos de procesamiento de señales a un entorno embebido permite analizar su comportamiento bajo restricciones reales, contribuyendo a una mejor comprensión de las implicaciones de su implementación y fortaleciendo el proceso de aprendizaje y validación de estas técnicas en el ámbito académico y tecnológico [1].

1.2.3. Enunciado del problema

Los procesos de modulación y demodulación suelen depender de simuladores o equipos especializados, y son limitadas las implementaciones accesibles que permiten ejecutar técnicas básicas como AM y ASK/OOK de forma eficiente sobre hardware embebido de bajo costo. Esta situación restringe la experimentación práctica y dificulta la evaluación del comportamiento real de estas técnicas en entornos académicos e investigativos.

1.3. Objetivos del proyecto

1.3.1. Objetivo general

Implementar un sistema embebido para la modulación y demodulación analógica y digital (AM y ASK/OOK), con el fin de que la evaluación de su desempeño en tiempo real se realice dentro de un entorno con recursos computacionales limitados, mediante la integración de procesos internos de generación, procesamiento y transmisión de señales.

1.3.2. Objetivos específicos

1. Integrar una versión simplificada de los algoritmos de modulación y demodulación en el sistema embebido, para el funcionamiento consistente de las técnicas en tiempo real, mediante ajustes en parámetros internos y estructuras de procesamiento.
2. Crear un sistema para la visualización y el análisis de los resultados de modulación y demodulación, mediante el envío estructurado de bloques de señal por comunicación serial hacia un computador externo.
3. Optimizar el procesamiento y flujo de datos de las señales dentro del dispositivo embebido, para un desempeño estable en condiciones de operación, mediante la disminución de la carga computacional.
4. Evaluar la calidad de los procesos de modulación y demodulación ejecutados en el sistema embebido, para la obtención de métricas de precisión y estabilidad, mediante el análisis comparativo de señales y parámetros de desempeño.

1.4. Alcances, entregables y limitaciones del proyecto

1.4.1. Alcances

El proyecto contempla la implementación de un sistema embebido capaz de ejecutar en tiempo real técnicas de modulación y demodulación analógicas y digitales (AM y ASK/OOK) sobre un microcontrolador de recursos limitados. Este sistema integra el procesamiento de señales dentro del dispositivo, permitiendo la generación, modulación y demodulación de señales bajo condiciones controladas.

Asimismo, se incluye el desarrollo de un mecanismo de comunicación entre el sistema embebido y un computador externo, con el objetivo de transmitir los datos generados para su visualización y análisis. Esto permite observar el comportamiento de las señales procesadas mediante una aplicación orientada a su representación gráfica.

Adicionalmente, el alcance considera la optimización del procesamiento y flujo de datos dentro del sistema embebido, así como la definición de métricas que permitan evaluar de manera objetiva el desempeño de los procesos implementados en términos de precisión y estabilidad.

1.4.2. Entregables

Como resultado del desarrollo del proyecto, se establece un conjunto de entregables que evidencian el cumplimiento de los objetivos planteados. En primer lugar, se obtiene un sistema embebido funcional capaz de ejecutar técnicas de modulación y demodulación (AM y ASK/OOK) en tiempo real.

Asimismo, se entrega el código embebido optimizado que implementa los algoritmos de procesamiento de señales y el manejo de la comunicación de datos. Este desarrollo se complementa con una aplicación en computador personal que permite la visualización y análisis de las señales generadas.

Adicionalmente, se define un protocolo de comunicación serial para la transmisión estructurada de datos entre el sistema embebido y la aplicación externa. Finalmente, se entrega el informe técnico que documenta el diseño, la implementación y la evaluación del sistema desarrollado.

1.4.3. Limitaciones

El desarrollo del proyecto se encuentra sujeto a limitaciones inherentes tanto al entorno de implementación como a las restricciones del hardware utilizado. En primer lugar, el sistema se desarrolla en un entorno académico controlado, por lo que no se contempla su validación en escenarios industriales ni en medios físicos reales de transmisión.

Desde el punto de vista técnico, la implementación se limita a técnicas de modulación y demodulación de baja complejidad, específicamente AM y ASK/OOK. La incorporación de técnicas adicionales depende de la capacidad del microcontrolador, particularmente en términos de procesamiento, memoria y velocidad de comunicación.

Adicionalmente, la visualización y el análisis de las señales se realizan mediante una aplicación externa en un computador personal, por lo que el sistema embebido no incorpora una interfaz gráfica propia. Finalmente, el procesamiento de señales se realiza de forma completamente digital dentro del dispositivo, sin considerar interacción directa con señales analógicas reales en medios de transmisión físicos.

Capítulo 2

Marco teórico

El análisis y procesamiento de señales constituye un elemento fundamental dentro de la ingeniería de computadores, particularmente en el diseño de sistemas digitales capaces de representar, transformar y transmitir información. En este contexto, la comprensión de los principios que rigen el comportamiento de las señales resulta indispensable para el desarrollo de técnicas que permitan su manipulación de manera controlada.

De manera análoga, cuando el tratamiento de señales se aborda dentro del desarrollo de soluciones en entornos computacionales específicos, surgen consideraciones adicionales que deben ser tomadas en cuenta para su correcta aplicación. Estas condiciones introducen nuevos elementos en el análisis, los cuales difieren de aquellos presentes en entornos de desarrollo más generales, haciendo necesario ampliar la base conceptual desde la cual se estudian estos procesos.

En el presente capítulo se establece el conjunto de fundamentos teóricos necesarios para comprender el tratamiento de señales, su transformación y las consideraciones asociadas a su implementación en sistemas digitales. A partir de esta base, se construye un marco conceptual que permite abordar de manera estructurada los elementos que intervienen en el desarrollo del problema de estudio.

2.1. Mapa conceptual del marco teórico

Con el fin de organizar de manera estructurada los conceptos desarrollados en este capítulo, se presenta el mapa conceptual mostrado en la Figura 2.1. Este permite visualizar la relación entre los distintos elementos teóricos y la progresión lógica bajo la cual serán abordados.

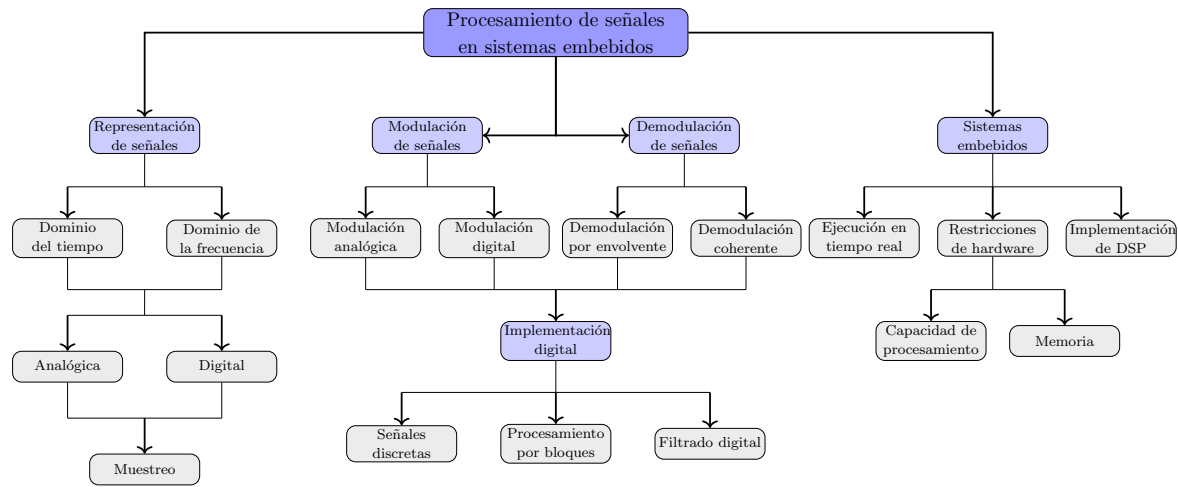


Figura 2.1: Mapa conceptual del marco teórico. Fuente: Elaboración propia.

Tal como se observa en la Figura 2.1, el desarrollo conceptual se estructura a partir de los fundamentos de procesamiento de señales, donde se establecen las bases para la representación de señales en distintos dominios. A partir de esta base, se introduce el proceso de modulación, considerando tanto enfoques analógicos como digitales, seguido por los mecanismos asociados a la demodulación de dichas señales.

Posteriormente, se incorpora la etapa de implementación digital, en la cual se consideran aspectos relacionados con la representación discreta, el procesamiento por bloques y el filtrado digital. Finalmente, el marco conceptual se completa con el análisis de los sistemas embebidos, donde se integran las restricciones de hardware y los requerimientos de tiempo real. Esta secuencia establece una transición progresiva desde la caracterización de señales hasta su ejecución en entornos embebidos.

2.2. Fundamentos de procesamiento de señales

El procesamiento de señales se fundamenta en el estudio de las representaciones matemáticas de la información y en las operaciones que pueden realizarse sobre estas para su análisis y transformación. Según Oppenheim y Willsky, en la teoría de Señales y Sistemas, este campo aborda la caracterización, manipulación y extracción de información contenida en señales mediante modelos matemáticos formales [2]. En el contexto de sistemas digitales, este estudio permite comprender cómo la información es modelada, manipulada y posteriormente utilizada dentro de distintos procesos computacionales.

2.2.1. Definición de señal

En este contexto, el concepto de señal constituye el punto de partida para el análisis, ya que permite formalizar la manera en que la información es representada dentro de un sistema.

De acuerdo con [2], una señal puede entenderse como una función que describe la variación de una magnitud en relación con una o más variables independientes. En la mayoría de los sistemas de interés en ingeniería, dicha variable corresponde al tiempo, lo que permite representar la evolución de la información dentro de un sistema.

Una señal continua puede representarse de forma general como:

$$x(t) \quad (2.1)$$

donde x denota la señal y t representa la variable independiente asociada al tiempo. Esta formulación permite modelar fenómenos físicos y procesos de información que varían de manera continua, tales como señales eléctricas o acústicas presentes en sistemas electrónicos [2].

Por otra parte, cuando la señal es tratada en sistemas digitales, su representación se realiza de forma discreta mediante una secuencia de valores definidos en instantes específicos. En este caso, la señal puede expresarse como:

$$x[n] \quad (2.2)$$

donde n es un índice entero que identifica cada muestra dentro de la secuencia. Esta representación es fundamental en el procesamiento digital, ya que permite manipular la señal mediante algoritmos y estructuras propias de sistemas computacionales [2].

Con el fin de complementar la representación matemática de las señales, es necesario considerar las magnitudes fundamentales que permiten caracterizar su comportamiento. En la Figura 2.2, se ilustran los principales parámetros asociados a una señal senoidal.

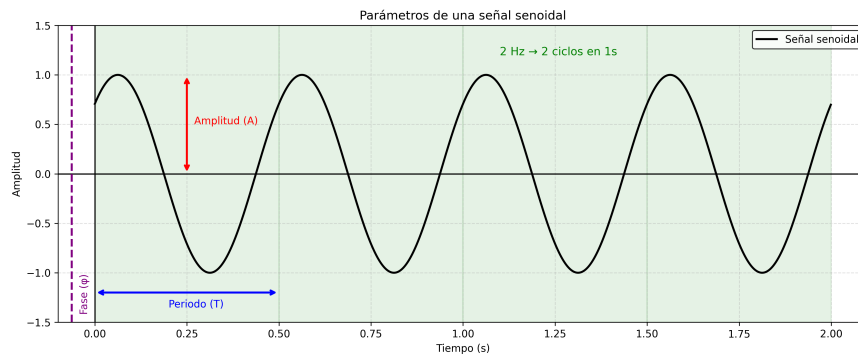


Figura 2.2: Parámetros fundamentales de una señal senoidal. Fuente: Elaboración Propia.

Tal como se muestra en la Figura 2.2, la amplitud corresponde al valor máximo que alcanza la señal respecto a su nivel de referencia, representando la intensidad de la magnitud descrita. El periodo, denotado como T , define el intervalo de tiempo en el cual la señal completa un ciclo. La frecuencia, representada como f , indica el número de ciclos por unidad de tiempo [9] y se relaciona con el periodo según:

$$f = \frac{1}{T} \quad (2.3)$$

Adicionalmente, la fase describe el desplazamiento relativo de la señal respecto a un origen de referencia en el tiempo, permitiendo identificar la posición inicial de la señal dentro de su ciclo. Estos parámetros constituyen la base para la descripción y análisis de señales en ingeniería, y serán utilizados de forma recurrente en el desarrollo de los procesos de modulación y demodulación abordados en las secciones posteriores.

2.2.2. Señales en el dominio del tiempo y la frecuencia

A partir de la representación de una señal, su análisis puede abordarse desde distintas perspectivas dependiendo de la información que se desee obtener. En este sentido, una misma señal puede ser descrita tanto en el dominio del tiempo como en el dominio de la frecuencia, lo que permite caracterizar su comportamiento desde enfoques complementarios.

En el dominio del tiempo, la señal se representa directamente como una función de la variable independiente, lo que permite observar su evolución a lo largo del tiempo. Esta representación resulta adecuada para analizar propiedades como la amplitud y la forma de la señal.

Por otra parte, el dominio de la frecuencia permite describir la señal en términos de las componentes espectrales que la conforman. De acuerdo con [2], cualquier señal puede expresarse como una combinación de componentes sinusoidales de distintas frecuencias.

La relación entre ambas representaciones se ilustra en la Figura 2.3. En el dominio del tiempo se observa una señal compuesta, cuya forma no permite identificar de manera directa las componentes que la originan. Sin embargo, al analizarla en el dominio de la frecuencia, se evidencian claramente las componentes sinusoidales presentes, representadas mediante picos en frecuencias específicas.

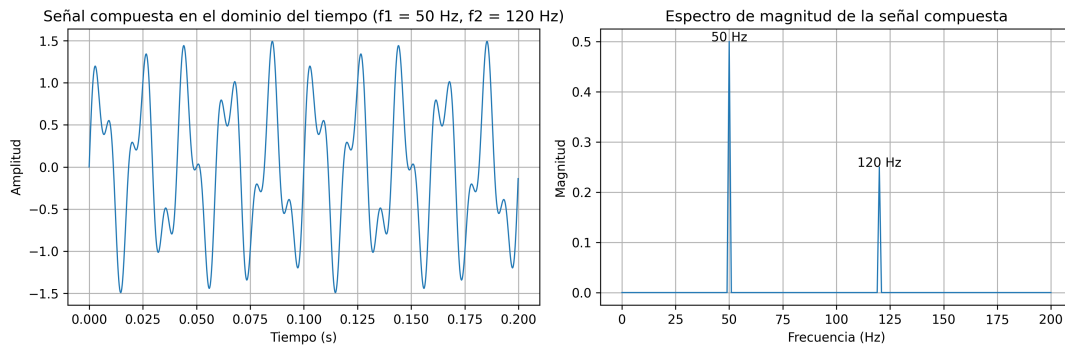


Figura 2.3: Representación de una señal compuesta en el dominio del tiempo y su correspondiente espectro de magnitud. Fuente: Elaboración Propia.

Según [2], la relación entre el dominio del tiempo y la frecuencia puede describirse mediante la Transformada de Fourier:

$$X(f) = \int_{-\infty}^{\infty} x(t)e^{-j2\pi ft}, dt \quad (2.4)$$

donde $X(f)$ representa la transformada de la señal $x(t)$, f corresponde a la frecuencia y j es la unidad imaginaria. Esta transformación permite describir cómo se distribuyen las componentes de la señal en función de la frecuencia.

La comprensión de estas dos representaciones resulta fundamental para el análisis de señales, ya que permite abordar tanto su comportamiento temporal como su composición espectral. A partir de esta base, es posible establecer una clasificación de las señales según la naturaleza de su representación.

2.2.3. Señales analógicas y digitales

Tal como se presenta en la Figura 2.1, una vez establecidas las formas de representación de las señales, resulta necesario considerar su clasificación según la naturaleza de los valores que adoptan y la forma en que son tratadas dentro de un sistema.

Según Lathi, en el estudio de sistemas de comunicación modernos, una señal analógica se caracteriza por tomar valores continuos tanto en el tiempo como en amplitud [9]. Este tipo de señal es común en fenómenos físicos, donde las magnitudes varían de manera continua, como es el caso de señales eléctricas, acústicas o electromagnéticas.

Por otra parte, una señal digital se define por tomar valores discretos en el tiempo y, en la mayoría de los casos, también en amplitud. Este tipo de representación es propia de los sistemas digitales, donde la información es tratada como una secuencia de valores que pueden ser almacenados, procesados y transmitidos mediante estructuras y algoritmos computacionales [10]. En aplicaciones prácticas, este tipo de señales es ampliamente utilizado en sistemas de comunicación y procesamiento implementados en plataformas digitales [5].

2.2.4. Muestreo y discretización

El tratamiento de señales en sistemas digitales requiere establecer un mecanismo que permita representar señales continuas mediante una forma discreta. En [2], el muestreo se define como el proceso mediante el cual una señal continua en el tiempo es evaluada en instantes específicos, generando una secuencia de valores que puede ser procesada digitalmente.

A partir de este proceso, la señal discreta puede expresarse como:

$$x[n] = x(nT_s) \quad (2.5)$$

donde T_s representa el período de muestreo y n es un índice entero que identifica cada muestra. La frecuencia de muestreo asociada se define de forma análoga a la relación entre frecuencia y periodo presentada en la Ecuación 2.3, obteniéndose:

$$f_s = \frac{1}{T_s} \quad (2.6)$$

El proceso de muestreo se ilustra en la Figura 2.4, donde se observa una señal continua junto con los valores discretos obtenidos al evaluarla en instantes uniformemente espaciados. En esta representación, las muestras corresponden a los valores de la señal en dichos instantes, evidenciando cómo la señal continua es representada mediante un conjunto finito de puntos.

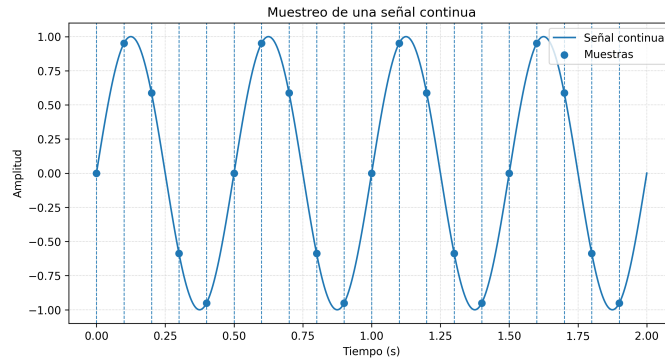


Figura 2.4: Ejemplo de muestreo de una señal continua. Fuente: Elaboración Propia.

Para que la señal original pueda ser representada adecuadamente a partir de sus muestras, es necesario que la frecuencia de muestreo cumpla con ciertos criterios. Según Shannon, en la teoría de comunicaciones, si se desea que una señal pueda ser reconstruida sin pérdida de información, la frecuencia de muestreo debe ser al menos el doble de la máxima frecuencia presente en la señal [11]. Este resultado es conocido como el teorema de Nyquist-Shannon.

De manera formal, esta condición puede expresarse como:

$$f_s \geq 2f_{max} \quad (2.7)$$

donde f_s corresponde a la frecuencia de muestreo y f_{max} representa la máxima frecuencia presente en la señal. El cumplimiento de esta condición garantiza que las componentes frecuenciales de la señal puedan ser representadas sin ambigüedad en el dominio discreto.

El análisis de estas condiciones resulta fundamental para comprender cómo el contenido de una señal se preserva durante su representación digital, estableciendo una relación directa con la forma en que puede ser caracterizado en términos de sus componentes en frecuencia.

2.2.5. Representación espectral

La representación de una señal en el dominio de la frecuencia permite describirla en términos de sus componentes espectrales, proporcionando una perspectiva basada en su contenido frecuencial. Tal como se ilustra en la Figura 2.3, el espectro de una señal representa la distribución de dichas componentes en función de la frecuencia.

A partir de la relación entre el dominio del tiempo y el dominio de la frecuencia, expresada previamente mediante la Transformada de Fourier en la ecuación (2.4), resulta posible identificar las frecuencias presentes en una señal y analizar cómo se distribuyen en términos de amplitud o energía, lo cual es fundamental para su caracterización en sistemas de procesamiento [9].

El espectro de magnitud, en particular, permite visualizar la intensidad de las componentes presentes en la señal, facilitando la identificación de frecuencias dominantes y estableciendo una base para su manipulación en función de sus características frecuenciales, como ocurre en procesos de modulación.

2.3. Modulación de señales

La modulación constituye uno de los procesos fundamentales en el tratamiento de señales dentro de los sistemas de comunicación. Su estudio permite comprender cómo una señal puede ser transformada para cumplir con los requisitos impuestos por un medio de transmisión, estableciendo así un vínculo entre la representación de la señal y su capacidad de propagación. En este sentido, la modulación se aborda como un mecanismo de transformación controlada de señales, cuyo análisis resulta esencial para el desarrollo de técnicas de transmisión eficientes.

2.3.1. Definición de modulación

El concepto de modulación ha sido abordado desde distintas perspectivas en la literatura. Según Haykin, en el contexto de los sistemas de comunicación, la modulación se define como el proceso mediante el cual una señal de información altera una propiedad de una señal de mayor frecuencia, con el propósito de facilitar su transmisión [1]. Por su parte, Proakis y Salehi plantean que la modulación no solo implica dicha interacción entre señales, sino que además permite trasladar el contenido de información hacia bandas de frecuencia específicas, favoreciendo la multiplexación y el aprovechamiento eficiente del espectro disponible [10].

Ambas definiciones coinciden en que la modulación implica una transformación de la señal original y, de manera complementaria, permiten entender este proceso tanto desde la interacción entre señales como desde su función dentro del sistema de comunicación. En conjunto, estas perspectivas establecen que la modulación no solo modifica características

de una señal, sino que también responde a condiciones específicas impuestas por el medio de transmisión.

En términos generales, la modulación puede interpretarse como un proceso mediante el cual una señal de información es representada a través de otra señal, modificando parámetros fundamentales como la amplitud, la frecuencia o la fase, lo que permite su adecuación a condiciones específicas de transmisión sin alterar el contenido esencial de la información.

Señal mensaje y señal portadora

Una vez definido el concepto de modulación, resulta necesario considerar las señales que intervienen en este proceso. En términos generales, la modulación se fundamenta en la interacción entre una señal mensaje y una señal portadora, cuya relación permite representar y trasladar la información dentro de un sistema de comunicación.

La señal mensaje, denotada como $m(t)$, corresponde a la señal que contiene la información que se desea transmitir. Su naturaleza depende del sistema considerado, ya que puede representar distintas magnitudes de interés, tales como audio, voz, datos o cualquier otra forma de información susceptible de ser transportada. En muchos casos, esta señal presenta componentes de baja frecuencia en comparación con la portadora, por lo que actúa como la fuente de información que será incorporada al proceso de modulación [9].

Por su parte, la señal portadora corresponde a una señal periódica de mayor frecuencia que actúa como soporte para la transmisión de la información contenida en la señal mensaje. Según [9], en los sistemas de comunicación modernos, esta señal suele representarse mediante una función sinusoidal, debido a que su comportamiento matemático facilita el análisis de los procesos de modulación. De forma general, la portadora puede expresarse como:

$$c(t) = A_c \cos(2\pi f_c t) \quad (2.8)$$

donde A_c corresponde a la amplitud de la portadora y f_c a su frecuencia.

La interacción entre $m(t)$ y $c(t)$ da lugar a la señal modulada, en la cual la información de la señal mensaje queda contenida en alguna de las propiedades de la portadora. En este sentido, la señal mensaje define el contenido informativo, mientras que la portadora establece la base sobre la cual dicho contenido puede ser transportado. Esta distinción resulta esencial para comprender las distintas técnicas de modulación desarrolladas en sistemas de comunicación.

2.3.2. Modulación analógica

La modulación analógica se caracteriza por operar sobre señales cuyo comportamiento es continuo en el tiempo y en amplitud. En este tipo de modulación, la señal mensaje modifica de manera proporcional alguna propiedad de la señal portadora, generando una señal modulada que conserva la naturaleza continua de la información original.

Según [1], las principales variantes de modulación analógica corresponden a la modificación de la amplitud, la frecuencia o la fase de la portadora. Por su parte, [9] destaca que estas técnicas surgen como respuesta a diferentes requerimientos del canal, tales como la eficiencia espectral o la robustez frente al ruido.

Esta clasificación permite entender que la modulación analógica no corresponde a una única técnica, sino a un conjunto de métodos que comparten un principio común, la representación continua de la información mediante la alteración de parámetros de una señal portadora.

De acuerdo con [1], en función del parámetro de la señal portadora que es modificado, es posible distinguir las siguientes técnicas principales:

AM: En la modulación en amplitud, la amplitud de la señal portadora varía en función de la señal mensaje, manteniendo constantes su frecuencia y fase.

FM: En la modulación en frecuencia, la frecuencia instantánea de la señal portadora se modifica en función de la señal de información, mientras que su amplitud permanece constante.

PM: En la modulación en fase, la variación se produce sobre la fase de la señal portadora en función de la señal mensaje, manteniendo constante su amplitud.

Cada una de estas técnicas presenta características particulares en términos de representación de la información y comportamiento frente al canal de transmisión, lo que permite su utilización en distintos escenarios de comunicación según los requerimientos del sistema.

Modulación en amplitud (AM)

De acuerdo con [10], la señal modulada en amplitud puede expresarse como:

$$s(t) = A_c (1 + \mu m(t)) \cos(2\pi f_c t) \quad (2.9)$$

donde A_c corresponde a la amplitud de la portadora, f_c a su frecuencia, μ representa el índice de modulación y $m(t)$ la señal mensaje normalizada, cumpliendo típicamente $m(t) \in [-1, 1]$. En esta expresión, el término $(1 + \mu m(t))$ actúa como una envolvente que define la variación de la amplitud de la señal portadora en función de la señal mensaje.

El concepto de envolvente, según [1], puede definirse como la curva que delimita los valores máximos y mínimos de la señal modulada, reflejando directamente el comportamiento de

la señal mensaje. De esta forma, la información contenida en $m(t)$ queda representada en la forma externa de la señal modulada, mientras que la portadora determina la oscilación de alta frecuencia.

El comportamiento de la señal AM se ilustra en la Figura 2.5, donde en el dominio del tiempo se observa la señal modulada junto con sus envolventes superior e inferior, las cuales siguen la forma de la señal mensaje.

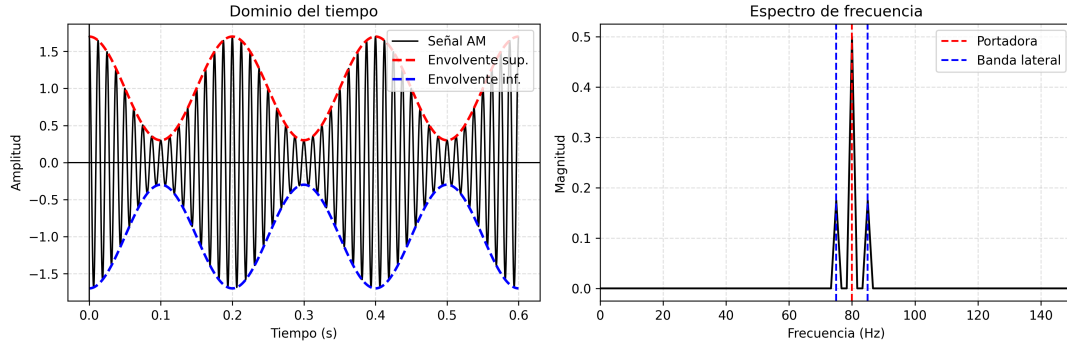


Figura 2.5: Señal modulada en amplitud (AM) en el dominio del tiempo y su representación en el dominio de la frecuencia. Fuente: Elaboración Propia.

Desde otra perspectiva, [9] señala que la modulación en amplitud puede interpretarse como un desplazamiento del espectro de la señal mensaje hacia una banda centrada en la frecuencia de la portadora. Tal como se muestra en la Figura 2.5, en el dominio de la frecuencia se evidencia la presencia de la componente de la portadora y de las bandas laterales, las cuales contienen la información de la señal mensaje. Esta representación permite comprender cómo la energía de la señal se distribuye alrededor de la frecuencia de la portadora como resultado del proceso de modulación.

Índice de modulación

A partir de la expresión de la señal modulada presentada en la ecuación (2.9), un parámetro relevante es μ , conocido como índice de modulación. Este cuantifica la relación entre la amplitud de la señal mensaje y la amplitud de la portadora, y se define como:

$$\mu = \frac{A_m}{A_c} \quad (2.10)$$

donde A_m corresponde a la amplitud máxima de la señal mensaje y A_c a la amplitud de la portadora. Este parámetro determina el grado en que la señal mensaje influye sobre la señal modulada.

Según [1], valores de μ menores que uno garantizan una modulación sin distorsión, mientras que valores superiores generan sobremodulación, lo que provoca la pérdida de información en la señal demodulada. En contraste, [9] enfatiza que el índice de modulación

no solo afecta la calidad de la señal, sino también la distribución de potencia entre la portadora y las bandas laterales.

Estas interpretaciones evidencian que el índice de modulación no es únicamente un parámetro matemático, sino un elemento clave en el análisis del comportamiento de la señal modulada.

2.3.3. Modulación digital

La modulación digital se fundamenta en la representación de la información mediante un conjunto discreto de símbolos, a diferencia de la modulación analógica donde la señal es continua. En este contexto, la señal mensaje se describe como una secuencia de valores discretos, cada uno de los cuales corresponde a un símbolo transmitido durante un intervalo de tiempo determinado.

En [10], se establece que cada símbolo puede asociarse a una forma de onda específica, permitiendo construir una señal modulada a partir de la concatenación de dichas formas de onda. Asimismo, en [1] se destaca que esta representación discreta favorece la robustez frente al ruido y habilita la aplicación de técnicas avanzadas de procesamiento digital.

De acuerdo con [10], en función del parámetro de la señal portadora que es modificado, es posible distinguir las siguientes técnicas principales:

ASK: En la modulación por desplazamiento de amplitud, la amplitud de la señal portadora toma valores discretos en función del símbolo transmitido.

FSK: En la modulación por desplazamiento de frecuencia, la frecuencia de la señal portadora cambia entre un conjunto discreto de valores según el símbolo transmitido.

PSK: En la modulación por desplazamiento de fase, la fase de la señal portadora adopta valores discretos que representan la información digital.

Estas técnicas comparten el principio de representar información discreta mediante la modificación de parámetros de una señal portadora, lo que constituye la base de los sistemas de comunicación digital.

Modulación digital por amplitud (ASK/OOK)

En la modulación por desplazamiento de amplitud (ASK), la amplitud de la señal portadora toma valores discretos en función del símbolo transmitido. A partir de la expresión de la señal portadora presentada en la ecuación (2.8), la señal modulada puede interpretarse como una variación discreta de su amplitud, manteniendo constante la frecuencia f_c . En este caso, el término A_c es reemplazado por un conjunto discreto de valores A_k , los cuales dependen directamente del símbolo transmitido.

En [10], se indica que ASK puede implementarse con múltiples niveles de amplitud, lo que permite representar más de un bit por símbolo. En su forma más simple, esta técnica se

reduce al caso binario conocido como On-Off Keying (OOK), donde la señal portadora se transmite únicamente cuando el símbolo corresponde a un valor lógico uno y se suprime en el caso de un cero. Este esquema puede interpretarse como una forma particular de ASK en la que la amplitud adopta únicamente dos valores posibles.

De acuerdo con Sklar, esta simplificación permite una implementación eficiente en sistemas digitales, aunque introduce limitaciones en términos de robustez frente al ruido, debido a la ausencia de portadora durante la transmisión de ciertos símbolos [12].

El comportamiento de la modulación OOK en el dominio del tiempo se ilustra en la Figura 2.6, donde se observa la correspondencia directa entre la señal binaria y la señal modulada, evidenciando la presencia de la portadora para valores lógicos uno y su ausencia para valores lógicos cero.

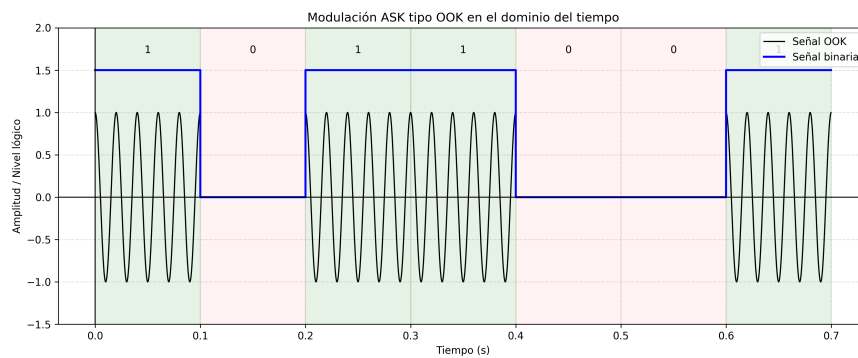


Figura 2.6: Modulación digital por amplitud tipo OOK en el dominio del tiempo. Fuente: Elaboración Propia.

2.4. Demodulación de señales

Una vez abordado el proceso de modulación y de acuerdo con la secuencia presentada en la Figura 2.1, tras el proceso de modulación resulta necesario considerar la etapa complementaria dentro del sistema de comunicación, encargada de procesar la señal recibida.

2.4.1. Definición de demodulación

El concepto de demodulación se establece como el proceso mediante el cual se extrae la señal de información a partir de una señal previamente modulada. Según Haykin, esta se define como la recuperación de la señal mensaje a partir de la señal modulada, mediante la inversión del mecanismo de modulación [1].

Asimismo, en [10] se plantea que la demodulación puede interpretarse como un proceso de estimación, en el cual se busca recuperar la información original considerando la posible presencia de perturbaciones introducidas por el canal de transmisión, tales como ruido y distorsión.

2.4.2. Demodulación de AM

Demodulación por detección de envolvente

La detección de envolvente constituye un método de demodulación no coherente, basado en la extracción directa de la envolvente de la señal modulada. En el caso de la modulación en amplitud, la envolvente de la señal sigue la forma de la señal mensaje, lo que permite recuperar la información sin requerir sincronización en fase con la portadora.

A partir de la expresión de la señal modulada definida en (2.9), se observa que la envolvente está dada por el término $A_c(1+\mu m(t))$, el cual contiene la información de la señal mensaje. En consecuencia, al extraer dicha envolvente se obtiene una versión escalada de $m(t)$. Este comportamiento puede observarse en la Figura 2.5, donde la envolvente superior e inferior delimitan la variación de la señal modulada en el dominio del tiempo.

De acuerdo con [1], este método presenta la ventaja de su simplicidad y de no requerir sincronización con la portadora. Sin embargo, su desempeño se ve limitado en condiciones donde existe sobremodulación o presencia significativa de ruido.

Demodulación coherente

Por otro lado, la demodulación coherente se basa en la multiplicación de la señal modulada por una señal portadora local que se encuentra sincronizada en frecuencia y fase con la portadora original. Este proceso permite trasladar el contenido espectral de la señal hacia bajas frecuencias, facilitando la recuperación de la señal mensaje.

Matemáticamente, este proceso puede expresarse como:

$$r(t) = s(t) \cdot \cos(2\pi f_c t) \quad (2.11)$$

donde $r(t)$ corresponde a la señal resultante del proceso de multiplicación, $s(t)$ representa la señal modulada y f_c define la frecuencia de la portadora. Al sustituir la expresión de $s(t)$ definida en (2.9), se obtiene una señal compuesta por componentes de baja y alta frecuencia. Mediante la aplicación de un filtro pasa bajas, es posible eliminar los términos de alta frecuencia y recuperar la señal mensaje.

Según [10], la demodulación coherente ofrece una mayor precisión en la recuperación de la señal en comparación con la detección de envolvente, especialmente en presencia de ruido. Sin embargo, requiere una sincronización precisa con la portadora, lo que incrementa la complejidad del sistema.

Filtrado en demodulación

El filtrado constituye un proceso fundamental en la recuperación de señales, ya que permite modificar el contenido espectral de una señal discreta con el objetivo de atenuar

o eliminar componentes de frecuencia no deseadas. En el contexto de la demodulación, este proceso resulta esencial para aislar la información de interés a partir de señales que contienen múltiples componentes en frecuencia.

De acuerdo con [2], un filtro puede describirse mediante una ecuación en diferencias de la forma:

$$y[n] = \sum_{k=0}^M b_k x[n-k] - \sum_{k=1}^N a_k y[n-k] \quad (2.12)$$

donde $x[n]$ representa la señal de entrada, $y[n]$ la señal de salida, y b_k , a_k corresponden a los coeficientes del sistema. Esta formulación permite modelar tanto filtros de respuesta finita como infinita, dependiendo de la estructura del sistema.

En procesos como la demodulación coherente, el filtrado se emplea para eliminar los componentes de alta frecuencia generados durante la multiplicación con la portadora, conservando únicamente las componentes de baja frecuencia asociadas a la señal mensaje. Este comportamiento puede observarse en la Figura 2.7, donde se muestra la señal posterior a la multiplicación junto con la señal filtrada, evidenciando la eliminación de las oscilaciones de alta frecuencia y la recuperación de la envolvente de baja frecuencia.

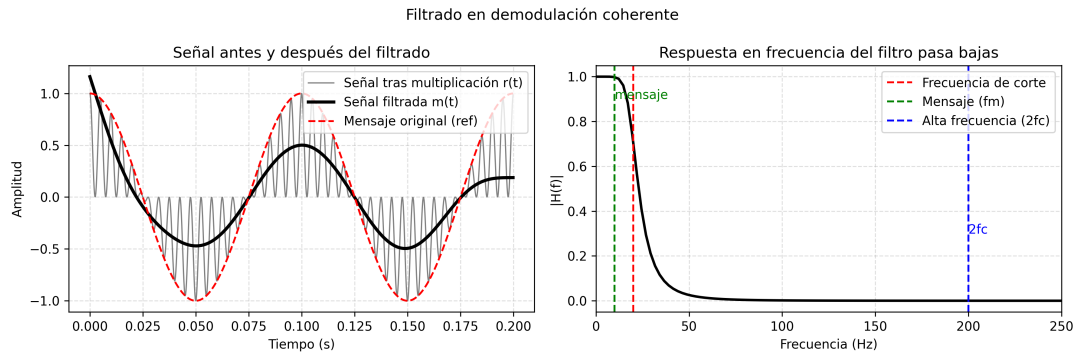


Figura 2.7: Proceso de filtrado en demodulación coherente. Fuente: Elaboración propia.

En este sentido, el filtro pasa bajas cumple una función fundamental, ya que permite el paso de las componentes espectrales en banda base mientras atenúa aquellas ubicadas en frecuencias superiores. Según Mitra, el comportamiento ideal de un filtro pasa bajas consiste en preservar completamente las frecuencias por debajo de una frecuencia de corte f_c y eliminar aquellas por encima de este valor [13]. No obstante, en implementaciones reales existe una transición gradual entre las bandas de paso y rechazo, lo que introduce efectos adicionales en la señal filtrada.

Estas consideraciones evidencian que el filtrado constituye un elemento esencial dentro del proceso de demodulación, al permitir separar la información útil de las componentes introducidas durante el procesamiento de la señal. Este principio resulta igualmente relevante en distintos esquemas de demodulación, independientemente de si la información se representa de forma continua o discreta.

2.4.3. Demodulación de señales digitales

En la modulación digital, la información se representa mediante un conjunto discreto de símbolos, por lo que el proceso de demodulación consiste en determinar cuál de los posibles símbolos fue transmitido a partir de la señal recibida.

En este contexto, [10] describe la demodulación como un problema de decisión, en el cual se selecciona el símbolo más probable en función de la señal observada. Asimismo, [1] establece que este proceso implica la reconstrucción de la secuencia de símbolos mediante el análisis de la señal en intervalos de tiempo discretos.

Estas definiciones permiten establecer que la demodulación digital no se orienta a la reconstrucción directa de una forma de onda continua, sino a la identificación de los símbolos transmitidos, a partir de los cuales se puede reconstruir la señal de información original.

Recuperación de símbolos en señales ASK/OOK

En las técnicas de modulación por amplitud digital, como ASK y OOK, la información se encuentra codificada en los niveles de amplitud de la señal portadora. Por lo tanto, la recuperación de símbolos consiste en analizar la amplitud de la señal recibida durante cada intervalo de símbolo.

Según [9], este proceso implica observar la señal en instantes específicos y comparar su amplitud con valores de referencia asociados a cada símbolo posible. En el caso de OOK, donde existen únicamente dos estados, la señal presenta una estructura particularmente simple, en donde la presencia de la portadora se asocia a un símbolo lógico uno, mientras que su ausencia corresponde a un símbolo lógico cero.

Sin embargo, la presencia de ruido y distorsión puede alterar la amplitud de la señal recibida, lo que introduce incertidumbre en el proceso de recuperación. Esto hace necesario definir criterios de decisión adecuados para garantizar una correcta interpretación de los símbolos.

Umbral de decisión

El umbral de decisión (threshold) es un criterio utilizado para determinar el símbolo transmitido a partir de la señal recibida. En el caso de modulación binaria como OOK, este proceso consiste en comparar la amplitud de la señal con un valor de referencia.

Si se denota la señal recibida como $r(t)$, el proceso de decisión puede representarse como:

$$\hat{b} = \begin{cases} 1, & \text{si } r(t) \geq \gamma \\ 0, & \text{si } r(t) < \gamma \end{cases} \quad (2.13)$$

donde γ representa el umbral de decisión (threshold) y $\hat{b}$ el símbolo estimado. Este criterio permite clasificar la señal en uno de los posibles valores discretos.

De acuerdo con [10], la selección del umbral adecuado resulta fundamental para minimizar errores en la detección, especialmente en presencia de ruido. Por su parte, [1] complementa señalando que el umbral óptimo depende de las características de la señal y del tipo de transmisión.

Estas consideraciones permiten establecer que el proceso de demodulación digital en esquemas como ASK y OOK se fundamenta en la evaluación de la señal recibida mediante criterios de decisión, los cuales condicionan directamente la correcta recuperación de los símbolos transmitidos.

2.5. Sistemas embebidos para procesamiento de señales

De acuerdo con la secuencia presentada en la Figura 2.1, tras abordar los fundamentos del procesamiento de señales, así como los procesos de modulación y demodulación, resulta necesario considerar el entorno en el cual estos algoritmos son implementados. En este contexto, los sistemas embebidos constituyen una plataforma fundamental para la ejecución de algoritmos de procesamiento de señales en entornos físicos, permitiendo trasladar los modelos teóricos hacia aplicaciones reales [14].

2.5.1. Definición de sistema embebido

El concepto de sistema embebido ha sido abordado desde diversas perspectivas. Según Wolf, un sistema embebido puede definirse como un sistema computacional diseñado para realizar una función específica dentro de un sistema mayor, operando bajo restricciones de recursos y con un comportamiento generalmente determinado por su aplicación [14].

Por otro lado, Noergaard plantea que los sistemas embebidos se caracterizan por la integración de hardware y software en un entorno optimizado para cumplir tareas particulares, destacando la estrecha relación entre ambos componentes [15].

Estas definiciones coinciden en que un sistema embebido no es un sistema de propósito general, sino una solución orientada a una función específica, lo que implica que su diseño y operación están fuertemente condicionados por los requerimientos de la aplicación.

2.5.2. Restricciones de hardware

Los sistemas embebidos operan bajo limitaciones inherentes a su arquitectura, las cuales influyen directamente en la implementación de algoritmos de procesamiento de señales. Entre las principales restricciones se encuentran la capacidad de procesamiento, la memoria disponible y la precisión numérica.

De acuerdo con [14], la capacidad de procesamiento determina la cantidad de operaciones que pueden ejecutarse en un intervalo de tiempo determinado, lo que condiciona la complejidad de los algoritmos que pueden ser implementados. En paralelo, la memoria disponible limita el tamaño de los datos y estructuras que pueden manejarse, afectando directamente estrategias como el procesamiento por bloques, el cual consiste en segmentar la señal en conjuntos finitos de muestras para su procesamiento independiente, permitiendo manejar señales de longitud arbitraria dentro de arquitecturas con memoria finita. Según Smith, este enfoque resulta eficiente desde el punto de vista computacional y se adapta naturalmente a sistemas con recursos limitados [16].

Por su parte, [13] señala que la precisión numérica, especialmente en sistemas de punto fijo, introduce errores asociados a la representación de valores. Estos errores, provocados por procesos como la cuantización, el redondeo y la saturación, generan desviaciones respecto al valor real de la señal, las cuales pueden acumularse a lo largo de múltiples operaciones y degradar progresivamente la calidad del procesamiento [2].

Estas restricciones evidencian que la implementación de técnicas de procesamiento de señales en sistemas embebidos no depende únicamente del modelo teórico, sino también de las capacidades del entorno en el cual se ejecutan, lo que obliga a establecer compromisos entre precisión numérica, uso de memoria y complejidad computacional.

2.5.3. Procesamiento en tiempo real

El procesamiento en tiempo real es una característica fundamental en muchos sistemas embebidos, especialmente aquellos orientados al tratamiento de señales. Un sistema se considera en tiempo real cuando es capaz de procesar la información dentro de un intervalo de tiempo determinado, cumpliendo con restricciones temporales estrictas.

De acuerdo con Kopetz, los sistemas en tiempo real se caracterizan no solo por la corrección de sus resultados, sino por la capacidad de producirlos dentro de un intervalo de tiempo previamente definido, lo que introduce una restricción temporal directa en el diseño de los algoritmos [17]. En el caso del procesamiento de señales, esto implica que operaciones como la modulación, demodulación y filtrado deben ejecutarse a una velocidad suficiente para seguir el ritmo de adquisición de datos, lo que impone límites sobre la complejidad de los algoritmos utilizados.

En síntesis, a lo largo de este capítulo se han establecido los fundamentos del procesamiento de señales, abordando su representación, así como los principios de modulación y demodulación tanto en esquemas analógicos como digitales. Adicionalmente, se han considerado los elementos asociados a su implementación en sistemas embebidos, destacando la influencia de las restricciones de hardware y de tiempo real sobre dichos procesos. Estos conceptos constituyen la base teórica sobre la cual se desarrollarán los algoritmos y técnicas que serán implementados en el contexto de este trabajo.

Capítulo 3

Marco metodológico

El desarrollo de sistemas embebidos orientados al procesamiento de señales requiere la adopción de un enfoque metodológico que permita gestionar de forma estructurada la interacción entre hardware y software, así como las restricciones de operación en tiempo real. En este contexto, la solución del problema se plantea mediante una estrategia que prioriza la descomposición del sistema en etapas funcionales, facilitando el análisis, diseño y validación progresiva de cada componente. De acuerdo con Heath [3], este tipo de sistemas demanda una integración controlada de sus elementos para garantizar coherencia en su comportamiento global, lo cual fundamenta la selección de un enfoque metodológico estructurado. En el presente capítulo se describe el marco metodológico adoptado, detallando la estrategia general de solución, la definición de las etapas del proceso, las consideraciones de implementación, los criterios de validación y verificación del sistema.

3.1. Estrategia general de solución

A partir del problema definido en la Sección 1.2, el cual establece la necesidad de implementar un sistema embebido capaz de ejecutar técnicas de modulación y demodulación analógica y digital bajo restricciones de operación reales, se plantea la adopción de una estrategia metodológica que permita estructurar de manera coherente el proceso de solución. En este contexto, la definición del enfoque de desarrollo resulta fundamental para garantizar que las decisiones de diseño se mantengan alineadas con los requerimientos del sistema y las limitaciones propias del entorno embebido.

3.1.1. Enfoque metodológico adoptado

La metodología de desarrollo adoptada es la iterativo-incremental, esta metodología está orientada a la construcción progresiva de sistemas mediante ciclos sucesivos de desarrollo, donde cada iteración incorpora nuevas funcionalidades o mejoras parciales sobre una versión previa del sistema. Según Pressman [18], este enfoque combina características del

desarrollo secuencial y evolutivo, permitiendo que el producto se construya de manera gradual a través de incrementos funcionales que son analizados, implementados y evaluados de forma continua.

Dentro de este modelo, cada iteración contempla un conjunto de etapas fundamentales que incluyen comunicación, planificación, modelado, construcción y despliegue. La etapa de comunicación se enfoca en la identificación de necesidades y requerimientos del sistema. Posteriormente, la planificación establece las tareas y objetivos asociados al incremento correspondiente. El modelado comprende las actividades de análisis y diseño necesarias para estructurar la solución propuesta. Luego, durante la construcción, se desarrollan las funciones del sistema y se realizan pruebas para validar su funcionamiento. Finalmente, el despliegue permite obtener una versión funcional parcial que puede ser evaluada y retroalimentada antes de iniciar el siguiente ciclo de desarrollo [18].

Una de las principales características de esta metodología consiste en que el sistema no se construye completamente en una única etapa, sino que evoluciona mediante incrementos funcionales sucesivos. Esto permite detectar problemas de manera temprana, realizar ajustes progresivos y validar continuamente el comportamiento del sistema conforme avanza el desarrollo. Adicionalmente, el enfoque iterativo-incremental favorece la organización modular del proceso de ingeniería y facilita la incorporación gradual de mejoras dentro del sistema.

La Figura 3.1 presenta una representación general del modelo iterativo-incremental descrito por Pressman [18]. En ella se observa cómo cada iteración atraviesa un conjunto de etapas similares, generando al final de cada ciclo un incremento funcional del sistema que sirve como base para el siguiente proceso de desarrollo.

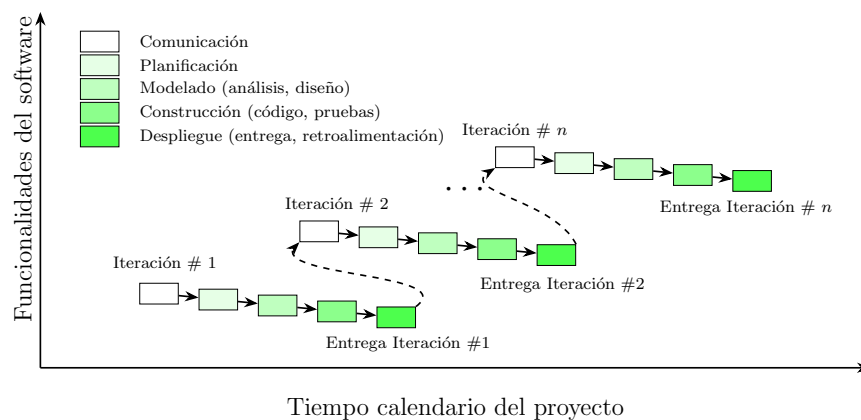


Figura 3.1: Representación del enfoque iterativo-incremental. Fuente: Adaptado de Pressman [18].

3.1.2. Enfoque técnico adoptado

Como complemento al enfoque metodológico iterativo-incremental descrito anteriormente, el desarrollo del sistema se plantea mediante una estrategia de co-diseño hardwa-

re/software. Este enfoque establece un proceso orientado al desarrollo coordinado de ambos dominios, considerando desde etapas tempranas la interacción existente entre el procesamiento ejecutado por software y las restricciones impuestas por el hardware. De acuerdo con [3], el co-diseño hardware/software permite estructurar el desarrollo de sistemas embebidos mediante una distribución funcional entre ambos dominios, facilitando la integración progresiva de los componentes del sistema y reduciendo problemas asociados a desacoplamientos durante etapas finales de implementación.

Bajo este enfoque, el proceso inicia con la definición del problema y la especificación del sistema, seguido de una etapa de particionamiento donde se asignan responsabilidades específicas al hardware y al software conforme a los requerimientos funcionales y restricciones de operación. Posteriormente, el desarrollo de ambos dominios se realiza de manera paralela, permitiendo que las decisiones asociadas al sistema puedan ajustarse progresivamente conforme avanza el desarrollo. Finalmente, los componentes desarrollados convergen en etapas de integración y validación, donde se verifica el comportamiento global del sistema y la interacción entre sus distintos elementos.

La Figura 3.2 presenta una representación general del proceso de co-diseño hardware/software, donde se observan las etapas de planificación, seguido de la separación funcional entre los dominios de hardware y software, así como las etapas de integración y validación utilizadas para verificar el comportamiento conjunto del sistema.

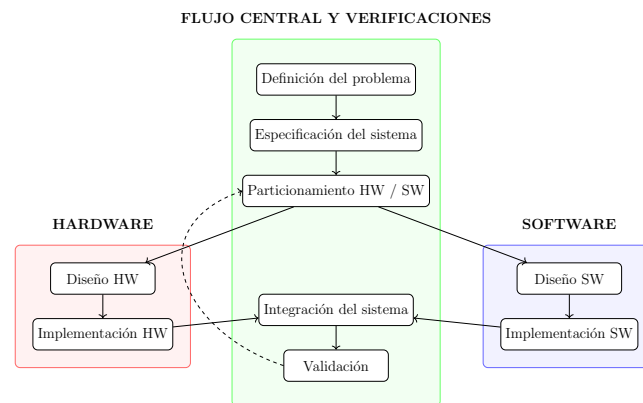


Figura 3.2: Proceso general de co-diseño hardware/software. Fuente: Elaboración propia.

3.1.3. Justificación del enfoque

La adopción de una metodología iterativo-incremental responde a la necesidad de desarrollar el sistema de manera progresiva, permitiendo incorporar y validar funcionalidades conforme avanza el proceso de desarrollo. En el contexto de la problemática planteada en la Sección 1.2, la implementación de técnicas de modulación y demodulación sobre un entorno embebido involucra ciertas restricciones, aspecto que requiere de un proceso continuo de ajuste y validación durante el desarrollo del sistema.

A diferencia de enfoques secuenciales tradicionales, donde la integración y verificación del

sistema se realizan principalmente en etapas finales, el desarrollo iterativo-incremental permite detectar limitaciones de manera temprana y realizar modificaciones progresivas sobre la solución conforme se identifican nuevos requerimientos o restricciones de operación. Esto resulta particularmente relevante en sistemas donde el comportamiento final depende de la interacción conjunta de distintos componentes.

Como complemento a esta metodología, el co-diseño hardware/software permite adaptar el enfoque iterativo-incremental a las necesidades propias del desarrollo de sistemas embebidos, donde el comportamiento final depende directamente de la interacción entre hardware y software. La incorporación de este enfoque facilita que cada iteración del desarrollo contemple de manera conjunta las restricciones y necesidades presentes en ambos dominios, permitiendo evaluar progresivamente el impacto de las decisiones tomadas sobre el funcionamiento global del sistema.

En enfoques centrados exclusivamente en software, las decisiones de diseño suelen asumirse bajo disponibilidad suficiente de recursos computacionales, condición que no necesariamente se cumple dentro del contexto embebido planteado. De forma similar, un enfoque centrado únicamente en hardware tiende a priorizar la eficiencia de ejecución, reduciendo la flexibilidad para adaptar o modificar los algoritmos y estructuras de procesamiento implementadas.

La incorporación de un enfoque de co-diseño permite mitigar estas limitaciones mediante un proceso coordinado entre ambos dominios, facilitando que las decisiones relacionadas con procesamiento, comunicación y ejecución puedan evaluarse de manera conjunta conforme avanza el desarrollo. Esto favorece una mejor adaptación del sistema a las restricciones reales de operación y contribuye a mantener coherencia entre los requerimientos funcionales y las capacidades del entorno embebido.

3.2. Estrategia de implementación

A partir de la metodología iterativo-incremental y del enfoque de co-diseño hardware/software definidos previamente, se establece una estrategia de implementación orientada al desarrollo progresivo del sistema integrando ambas estrategias. De manera que, cada iteración contempla un conjunto de etapas que permiten definir objetivos, desarrollar funcionalidades, integrar componentes y validar el comportamiento del sistema conforme avanzan las distintas fases del proyecto.

Dentro de este proceso, el enfoque iterativo-incremental proporciona la estructura general de evolución del sistema, mientras que el co-diseño hardware/software permite coordinar el desarrollo de ambos dominios durante las etapas técnicas asociadas a diseño e implementación. De esta manera, cada incremento funcional del sistema incorpora procesos de análisis, desarrollo, integración y validación que facilitan el ajuste progresivo de la solución conforme se identifican nuevas restricciones o necesidades de operación.

La Figura 3.3 presenta el proceso metodológico adoptado para el desarrollo del proyecto.

En ella se observa la organización general de las etapas que conforman cada iteración, así como la interacción existente entre los dominios de hardware y software durante la etapa de desarrollo.

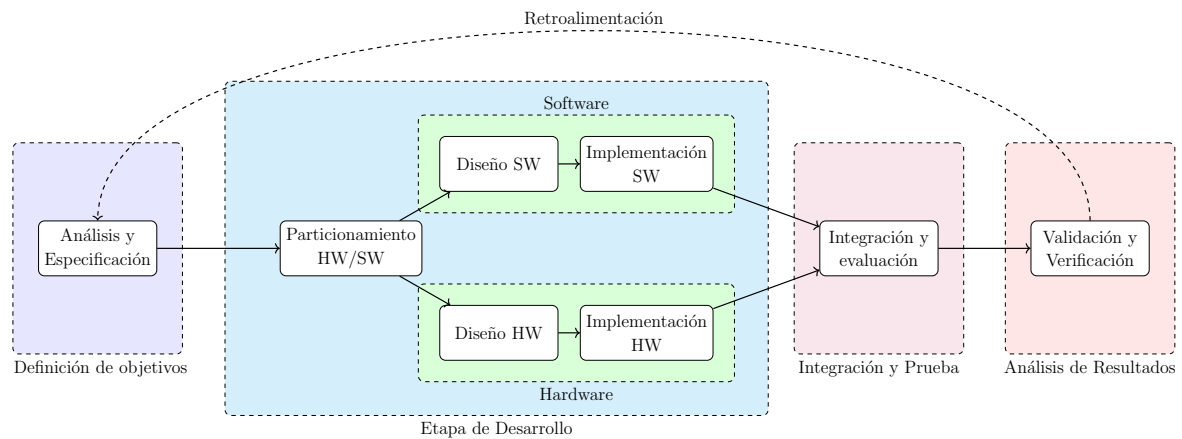


Figura 3.3: Proceso metodológico para el desarrollo del sistema. Fuente: Elaboración propia.

3.2.1. Descripción de las etapas del proceso

El proceso metodológico se organiza en etapas que permiten estructurar el desarrollo progresivo del sistema conforme avanzan las distintas iteraciones. Cada una de estas etapas responde a objetivos específicos dentro del proceso general y mantiene una relación directa con las necesidades funcionales y restricciones asociadas al proyecto. A continuación, se describen las etapas que conforman el proceso.

Definición de objetivos: En esta etapa se establecen los requerimientos y objetivos correspondientes a cada iteración del desarrollo. Se delimitan las funcionalidades que serán incorporadas o ajustadas dentro del sistema, así como las restricciones y criterios generales que orientan el proceso de implementación. Esta etapa permite mantener una evolución controlada del sistema conforme avanzan las iteraciones.

Etapla de desarrollo: Corresponde a la fase principal de construcción del sistema y se encuentra organizada bajo un enfoque de co-diseño hardware/software. Inicialmente, se realiza un proceso de particionamiento funcional donde se distribuyen responsabilidades entre ambos dominios según las necesidades del sistema y las restricciones de operación identificadas. Posteriormente, las actividades de diseño e implementación de hardware y software se desarrollan de manera coordinada y paralela, permitiendo mantener coherencia entre procesamiento, ejecución e interacción de los distintos componentes del sistema.

Dentro de esta etapa se contemplan actividades asociadas a adaptación algorítmica, implementación de procesamiento, organización del flujo de datos y adecuación progresiva de los componentes del sistema conforme evolucionan los requerimientos de cada iteración.

Integración y pruebas: Una vez desarrollados los distintos componentes, se procede a su

integración con el objetivo de conformar una versión funcional del sistema. Durante esta etapa se verifica la interacción entre hardware y software, permitiendo identificar posibles inconsistencias derivadas de su acoplamiento y evaluar el comportamiento general del sistema bajo condiciones de operación.

Análisis de resultados: Posteriormente, se analizan los resultados obtenidos durante las pruebas realizadas sobre el sistema. Esta etapa permite determinar el grado de cumplimiento de los objetivos definidos para cada iteración, así como identificar ajustes necesarios relacionados con estabilidad, funcionamiento y desempeño general de la solución.

Retroalimentación: Finalmente, los resultados obtenidos durante el análisis son utilizados como insumo para las siguientes iteraciones del desarrollo. Esta etapa permite incorporar mejoras progresivas sobre el sistema, redefinir objetivos cuando resulte necesario y mantener un proceso continuo de ajuste y validación conforme evoluciona la solución propuesta.

3.3. Estrategia de validación y verificación

Como parte de las etapas de integración, pruebas y análisis de resultados descritas previamente dentro del proceso metodológico, se establece una estrategia de validación y verificación orientada a evaluar el cumplimiento de los objetivos definidos para cada iteración del desarrollo. Este proceso de validación se fundamenta en la definición de criterios de evaluación alineados con el problema abordado, de manera que no se limita únicamente a verificar las funcionalidades incorporadas durante cada iteración, sino también a garantizar que la integración progresiva de nuevos componentes y características mantenga la estabilidad, coherencia y funcionamiento general del sistema.

En primer lugar, se establece como criterio principal la capacidad de recuperación de la señal, entendida como la correspondencia entre la señal original y la señal obtenida tras el proceso de modulación y demodulación. Este criterio permite evaluar la calidad del procesamiento realizado mediante la aplicación de métricas de similitud y error entre señales.

De forma complementaria, resulta necesario verificar que el procesamiento de la información se realice dentro de intervalos de tiempo adecuados, con el objetivo de garantizar la integridad y continuidad del flujo de datos. Para ello, se consideran mecanismos de medición y análisis orientados a evaluar tiempos de respuesta y latencia durante la ejecución del sistema.

Adicionalmente, se incorpora como criterio la estabilidad del sistema, la cual se relaciona con la capacidad de mantener un comportamiento consistente ante variaciones en los parámetros de operación. Este aspecto permite identificar posibles degradaciones en el procesamiento y evaluar la robustez de la solución frente a condiciones de operación variables.

En conjunto, esta estrategia de validación y verificación permite evaluar el sistema desde una perspectiva funcional y de desempeño, asegurando que la solución desarrollada responde de manera coherente a los requerimientos y restricciones establecidos para el proyecto.

3.4. Herramientas y tecnologías consideradas

Para el desarrollo del proyecto se contempla el uso de distintas herramientas de software, bibliotecas, entornos de desarrollo y plataformas de apoyo. A continuación, se presentan las principales herramientas consideradas dentro del desarrollo de la solución.

- **Python 3.11:** Definido como entorno principal para el desarrollo del sistema de monitoreo y procesamiento. Dentro de este entorno se incorporan bibliotecas como NumPy para operaciones matemáticas y procesamiento numérico, PySerial para comunicación serial entre el sistema embebido y el computador, y JSON para la administración de archivos de configuración.
- **MicroPython:** Empleado como entorno de ejecución para el desarrollo de funciones embebidas, permitiendo el manejo de periféricos y la implementación de componentes asociados al procesamiento dentro del sistema.
- **PyQt5 y PyQtGraph:** Orientados al desarrollo de la interfaz gráfica y la visualización en tiempo real de señales y formas de onda dentro del sistema de monitoreo.
- **MATLAB:** Considerado como herramienta de apoyo para análisis matemático de señales, generación de gráficas y validación conceptual de ecuaciones relacionadas con técnicas de modulación, demodulación y métricas de evaluación.
- **Tinkercad:** Plataforma de apoyo para diseño preliminar de partes del sistema mediante el 3D Designer, así como para simulación del circuitos utilizando Tinkercad Circuit Simulator.
- **Entorno de desarrollo:** Compuesto principalmente por Windows 10 como sistema operativo base, Visual Studio Code como editor principal de programación y GitHub como plataforma de control de versiones y administración del código fuente.

3.5. Desglose del marco metodológico

Con el fin de establecer una correspondencia directa entre los objetivos específicos del proyecto y el proceso metodológico seguido, se presenta la Tabla 3.1, en la cual se detallan las principales actividades, entregables, herramientas y estrategias de validación asociadas a cada objetivo. Esta relación permite evidenciar la forma en que cada componente del desarrollo contribuye al cumplimiento de los resultados planteados.

Tabla 3.1: Desglose del marco metodológico por objetivos.

Objetivo	Actividades	Entregables	Herramientas	Validación
Integrar una versión simplificada de los algoritmos de modulación y demodulación en el sistema embebido, para el funcionamiento consistente de las técnicas en tiempo real, mediante ajustes en parámetros internos y estructuras de procesamiento.	Adaptación de algoritmos, ajuste de parámetros internos y validación progresiva del procesamiento.	Algoritmos integrados y funcionales dentro del entorno embebido.	Python 3.11, MicroPython y herramientas de análisis matemático y procesamiento de señales.	Verificación de recuperación de señal, estabilidad del procesamiento y comportamiento del sistema durante la ejecución en tiempo real.
Crear un sistema para la visualización y el análisis de los resultados de modulación y demodulación, mediante el envío estructurado de bloques de señal por comunicación serial hacia un computador externo.	Desarrollo del sistema de comunicación serial e integración con la interfaz de monitoreo.	Sistema de visualización y monitoreo funcional.	PyQt5, PyQt-Graph, PySerial y herramientas de comunicación serial.	Validación de integridad y continuidad del flujo de datos, así como coherencia visual entre las señales transmitidas y procesadas.
Optimizar el procesamiento y flujo de datos de las señales dentro del dispositivo embebido, para un desempeño estable en condiciones de operación, mediante la disminución de la carga computacional.	Adecuación del flujo de datos, reducción de carga computacional y ajuste progresivo de estructuras de procesamiento.	Sistema optimizado para operación estable dentro del entorno embebido.	Python, MicroPython, herramientas de monitoreo del sistema y mecanismos de análisis de tiempos de ejecución.	Evaluación de tiempos de respuesta (Latencia), continuidad del procesamiento y estabilidad del sistema bajo distintas condiciones de operación.
Evaluar la calidad de los procesos de modulación y demodulación ejecutados en el sistema embebido, para la obtención de métricas de precisión y estabilidad, mediante el análisis comparativo de señales y parámetros de desempeño.	Pruebas controladas, obtención de métricas y análisis comparativo entre señales.	Resultados de evaluación y métricas de desempeño del sistema.	Matlab, métricas de similitud y herramientas de análisis de señales.	Aplicación de métricas de similitud y error (NCC, RMSE etc.) entre señales para evaluar precisión, estabilidad y comportamiento general del sistema.

Capítulo 4

Descripción del trabajo realizado

El presente capítulo describe el proceso de desarrollo e integración de la solución propuesta para el sistema de monitoreo y análisis de técnicas de modulación y demodulación implementado en el proyecto. Se abordan las principales decisiones de ingeniería adoptadas durante la construcción de la solución, considerando aspectos asociados al procesamiento digital de señales, integración hardware/software, comunicación de datos, modularidad y restricciones propias del entorno utilizado.

Inicialmente, se presenta una descripción general de la arquitectura de la solución y de la interacción entre sus principales componentes funcionales. Posteriormente, se describe el proceso seguido para el desarrollo de cada uno de los objetivos específicos del proyecto, enfatizando los criterios técnicos considerados durante la implementación y refinamiento progresivo del sistema. Finalmente, se realiza un análisis de los resultados obtenidos a partir del comportamiento funcional de la solución.

4.1. Descripción de la arquitectura de la solución

La solución propuesta se estructuró a partir de una arquitectura modular orientada a separar las principales responsabilidades funcionales del sistema, incluyendo adquisición de entradas, procesamiento de señales, transmisión de datos y visualización de información. Esta organización permitió distribuir el procesamiento entre el sistema embebido y la aplicación de monitoreo, facilitando la integración entre ambos entornos y manteniendo una estructura de funcionamiento más organizada y escalable.

La Figura 4.1 presenta la arquitectura general de la solución propuesta y la interacción existente entre los principales módulos funcionales del sistema.

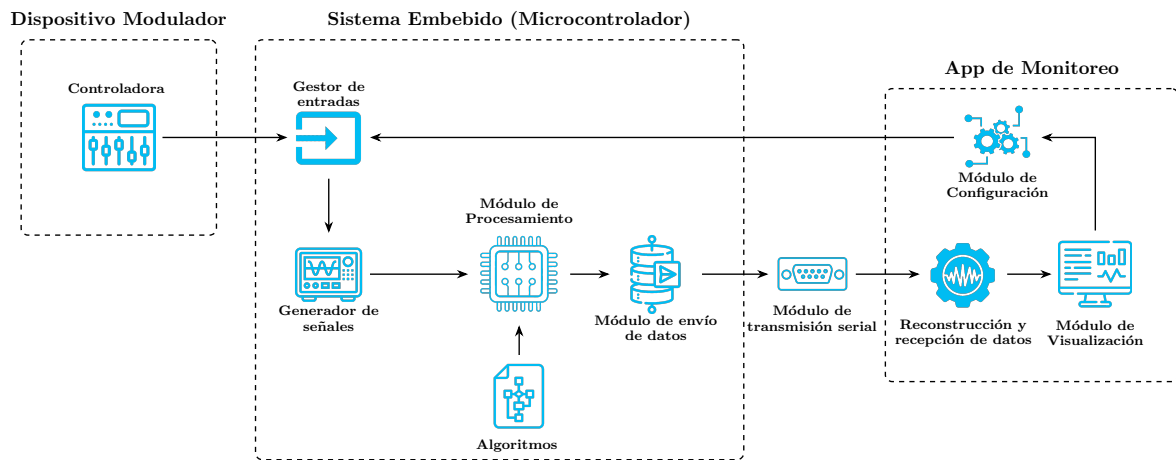


Figura 4.1: Diagrama de arquitectura de la solución. Fuente: Elaboración propia.

A nivel general, la arquitectura se compone de un dispositivo modulador basado en controles analógicos, un sistema embebido encargado del procesamiento principal de señales, un mecanismo de transmisión serial para el intercambio de información y una aplicación de monitoreo orientada a la reconstrucción, visualización y configuración del sistema. Dentro del sistema embebido se estableció una separación funcional entre los módulos de gestión de entradas, generación de señales, procesamiento de modulación y demodulación, integración de algoritmos y empaquetamiento de datos, permitiendo desacoplar las diferentes responsabilidades asociadas al flujo de procesamiento.

De manera complementaria, la aplicación de monitoreo se estructuró mediante módulos independientes de recepción y reconstrucción de datos, visualización de señales y configuración general del sistema, permitiendo mantener separadas las tareas asociadas al procesamiento gráfico, interpretación de información y control operativo.

4.2. Descripción del proceso de solución

A partir de la arquitectura definida para la solución propuesta, el proceso de desarrollo del sistema se orientó a la integración progresiva de los distintos componentes funcionales. A continuación, se describe el desarrollo realizado para cada uno de estos componentes, considerando las principales decisiones técnicas asociadas al sistema embebido, el procesamiento de señales, la comunicación de datos y la aplicación de monitoreo implementada para el proyecto.

4.2.1. Adaptación de los algoritmos al entorno embebido

La adaptación de los algoritmos de modulación y demodulación al entorno embebido representa uno de los principales retos técnicos de la solución propuesta, debido a que las técnicas implementadas deben mantener funcionamiento continuo bajo restricciones aso-

ciadas a procesamiento, memoria, sincronización temporal y transmisión serial de datos. En este escenario, el problema no se limita únicamente a ejecutar ecuaciones de modulación y demodulación, sino a reorganizar las técnicas de procesamiento para mantener estabilidad operativa dentro de una arquitectura computacionalmente limitada.

Selección de la plataforma embebida

La selección de la plataforma embebida condiciona directamente la complejidad matemática permitida, la organización del procesamiento y la viabilidad temporal del sistema. Bajo este criterio, la plataforma seleccionada debe proporcionar capacidad suficiente para ejecutar procesamiento digital básico de señales, transmisión serial continua y adquisición de entradas analógicas, manteniendo simultáneamente bajo costo y simplicidad de integración.

La Tabla 4.1 presenta una comparación entre algunas de las plataformas consideradas durante esta etapa, utilizando información proveniente de las especificaciones oficiales de cada fabricante [19].

Tabla 4.1: Comparación entre plataformas embebidas consideradas.

Plataforma	Frecuencia	RAM	GPIO	USB	Costo aprox.
Arduino Uno	16 MHz	2 kB	14	USB Serial	\$20
Raspberry Pi Pico	133 MHz	264 kB	26	USB CDC	\$4
ESP32	240 MHz	520 kB	34	USB-UART	\$8

En primera instancia, la plataforma Arduino Uno presenta restricciones importantes asociadas tanto a memoria como a capacidad de procesamiento para mantener generación, modulación, demodulación y transmisión continua de señales dentro de un mismo flujo operativo. La limitada cantidad de memoria SRAM disponible restringe significativamente el manejo simultáneo de buffers de señal, estructuras temporales de procesamiento y paquetes de transmisión serial, mientras que la frecuencia de operación del microcontrolador introduce limitaciones adicionales sobre la continuidad temporal requerida por el sistema. Bajo estas condiciones, la ejecución concurrente de procesos puede comprometer la estabilidad general del sistema.

Por otro lado, aunque ESP32 proporciona mayores capacidades computacionales y una cantidad superior de memoria disponible, gran parte de su arquitectura se orienta hacia aplicaciones IoT y comunicación inalámbrica. Esto incorpora subsistemas y mecanismos de conectividad que no aportan ventajas directas para el problema abordado, pero sí aumentan la complejidad general de integración, configuración y administración del entorno embebido. Adicionalmente, parte del enfoque de la solución consiste en mantener una arquitectura computacional relativamente simple, donde el procesamiento digital y

la sincronización temporal de señales representen el núcleo principal del sistema.

Seguidamente, la Raspberry Pi Pico representa una alternativa más balanceada entre capacidad computacional, disponibilidad de periféricos y costo de implementación. La plataforma proporciona capacidad suficiente para mantener procesamiento continuo de señales, lectura simultánea de entradas analógicas y transmisión serial estructurada sin incorporar complejidad adicional asociada a conectividad externa o subsistemas no requeridos. Adicionalmente, la integración directa de comunicación USB, la disponibilidad de GPIO suficientes para interacción física y la compatibilidad con MicroPython favorecen una arquitectura más unificada entre el entorno embebido y el sistema de monitoreo desarrollado.

Luego de evaluar las opciones disponibles, la Raspberry Pi Pico es la plataforma embebida seleccionada para el desarrollo de la solución propuesta, esta placa utiliza como núcleo principal el microcontrolador RP2040, cuyas características técnicas relevantes se presentan en la Tabla 4.2 [19].

Tabla 4.2: Características técnicas relevantes del RP2040.

Característica	Descripción
Arquitectura	Dual-Core Arm Cortex-M0+
Frecuencia máxima	Hasta 133 MHz
Memoria SRAM	264 kB
Memoria Flash	Externa mediante QSPI
GPIO disponibles	26 GPIO multipropósito
Entradas ADC	3 canales de 12 bits
Comunicación USB	USB 1.1 con soporte CDC
Lenguajes soportados	C/C++ y MicroPython

A pesar de que la Raspberry Pi Pico representa la alternativa más adecuada para los requerimientos del sistema, el entorno embebido continúa imponiendo restricciones importantes sobre la complejidad matemática permitida durante la ejecución continua del procesamiento. Las capacidades computacionales y de memoria del RP2040 limitan el uso de bibliotecas orientadas a procesamiento numérico intensivo, como NumPy o SciPy, debido a sus requerimientos de memoria y estructuras vectorizadas. En consecuencia, la arquitectura de procesamiento requiere apoyarse principalmente en operaciones aritméticas básicas, procesamiento secuencial y estructuras matemáticas simplificadas.

Integración física del modulador

Luego de seleccionar la plataforma embebida, se requiere definir un circuito que permita poner a disposición una estructura física mínima compuesta por elementos electrónicos que permitan interactuar dinámicamente con las técnicas que se ejecutarán en el embebido. Bajo este criterio, el sistema se construye utilizando una estructura compuesta por la Raspberry Pi Pico, tres potenciómetros y un interruptor.

La Figura 4.2 presenta el esquema general de conexión utilizado para la construcción del modulador físico.

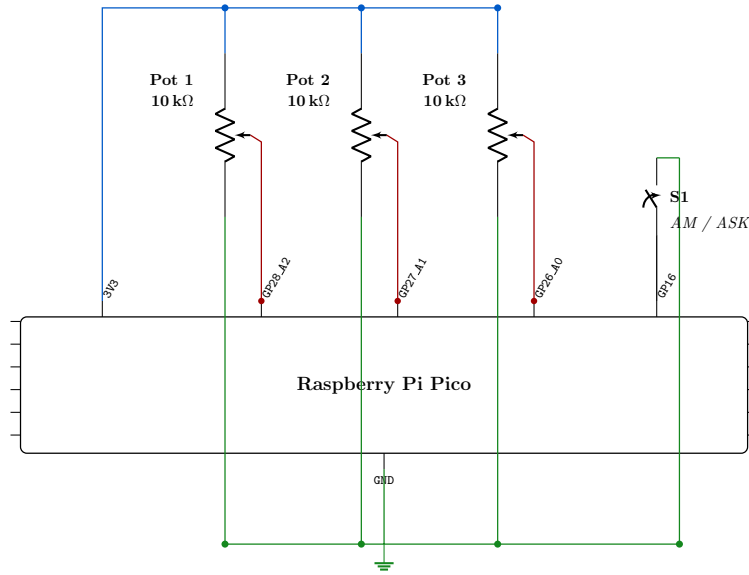


Figura 4.2: Esquemático general del módulo físico de control utilizado por el sistema embebido.

Adaptación de la técnica AM

La adaptación de la técnica AM al entorno embebido representa una decisión crítica dentro del diseño de la solución, debido a que esta técnica puede implementarse con distintos niveles de complejidad matemática y, por tanto, con diferentes implicaciones sobre precisión, carga computacional y estabilidad temporal. Bajo este escenario, el problema no consiste únicamente en aplicar la ecuación clásica de modulación, sino en definir hasta qué punto resulta conveniente trasladar al microcontrolador una representación más completa del proceso de modulación y demodulación, considerando que el sistema debe ejecutar simultáneamente generación de señales, lectura de parámetros físicos, procesamiento continuo de bloques y transmisión serial hacia la aplicación de monitoreo.

Para la etapa de modulación AM se consideran dos enfoques principales. El primero corresponde a una implementación con mayor acondicionamiento matemático sobre la señal, incorporando una etapa de normalización, control más estricto de amplitud y ajustes adicionales sobre la señal de información antes de aplicar la portadora. Bajo esta alternativa, la señal de información $m[n]$ no se utiliza directamente, sino que es previamente acondicionada para mantener su amplitud dentro de un rango controlado:

$$m_a[n] = \frac{m[n]}{\max(|m[n]|)} \quad (4.1)$$

Posteriormente, la señal modulada se obtiene mediante:

$$s[n] = A_c (1 + \mu m_a[n]) \cos(2\pi f_c n T_s) \quad (4.2)$$

donde A_c representa la amplitud de la portadora, μ el índice de modulación, f_c la frecuencia de portadora y T_s el periodo de muestreo.

Esta alternativa proporciona un mayor control matemático sobre la amplitud de la señal modulada y reduce el riesgo de sobre-modulación o saturación frente a variaciones de amplitud de la señal de entrada. Adicionalmente, el uso de una señal previamente acondicionada favorece un comportamiento más estable de la envolvente y contribuye a mantener una reconstrucción más controlada durante la demodulación.

Sin embargo, este enfoque también incrementa considerablemente la carga de procesamiento dentro del entorno embebido. La normalización requiere recorrer bloques completos de muestras, calcular máximos, realizar divisiones sucesivas y mantener controles adicionales sobre el rango dinámico de la señal. Aunque estas operaciones resultan relativamente comunes en entornos de simulación o procesamiento DSP convencional, su ejecución continua dentro de MicroPython sobre un microcontrolador implica un aumento significativo en el costo computacional. En consecuencia, esta alternativa prioriza precisión matemática y acondicionamiento de señal, pero aproxima simultáneamente al sistema embebido a sus límites de procesamiento.

Como alternativa, la modulación puede representarse mediante una formulación discreta simplificada orientada a preservar únicamente el comportamiento funcional esencial de AM. En este contexto, la señal de información es generada directamente dentro de un rango previamente controlado, eliminando la necesidad de incorporar una etapa adicional de normalización dinámica por bloque. La modulación se expresa como:

$$s[n] = (1 + \mu m[n]) c[n] \quad (4.3)$$

con:

$$c[n] = \cos(2\pi f_c n T_s) \quad (4.4)$$

Esta formulación reduce el procesamiento requerido a operaciones elementales de suma, multiplicación y evaluación trigonométrica de la portadora. La principal ventaja de este enfoque corresponde a la disminución significativa de la carga computacional asociada al procesamiento continuo. Debido a que la arquitectura elimina etapas adicionales de acondicionamiento y control dinámico, el microcontrolador dispone de mayor margen operativo para sostener simultáneamente el procesamiento interno, la transmisión serial y la actualización continua de parámetros del sistema.

La diferencia entre ambas alternativas radica en que, la primera estrategia favorece la precisión matemática, control de amplitud y estabilidad formal sobre la señal modulada, pero incrementa el procesamiento requerido por bloque, mientras que, la segunda reduce parte de ese acondicionamiento adicional y prioriza simplicidad operativa y estabilidad temporal dentro del entorno embebido, conservando al mismo tiempo el comportamiento funcional fundamental de la técnica.

Considerando que la señal de información utilizada dentro del sistema no proviene de una fuente externa arbitraria, sino que es generada internamente mediante parámetros controlados por el propio sistema, la arquitectura final se orienta hacia la formulación discreta simplificada. Esta decisión permite mantener una representación adecuada del fenómeno de modulación AM sin incorporar etapas adicionales de procesamiento que incrementen innecesariamente la carga computacional del microcontrolador. En consecuencia, el enfoque seleccionado proporciona un balance más adecuado entre representatividad funcional, estabilidad temporal y viabilidad computacional dentro del entorno embebido.

La etapa de demodulación introduce una segunda decisión crítica asociada al mecanismo de recuperación de la señal de información. En este caso se consideran dos alternativas técnicamente viables: la demodulación por envolvente y la demodulación coherente. Ambos enfoques permiten recuperar la información contenida en la señal AM, aunque presentan diferencias importantes respecto a sensibilidad, complejidad y compatibilidad con la arquitectura general del sistema.

La primera alternativa corresponde a la demodulación por envolvente. En este enfoque, la señal modulada es transformada en una señal positiva que permita aproximar su envolvente y posteriormente aplicar filtrado pasa bajos para recuperar la señal de información. De forma simplificada, este proceso puede representarse mediante:

$$e[n] = |s[n]| \quad (4.5)$$

seguido por:

$$\hat{m}[n] = LPF\{e[n]\} \quad (4.6)$$

La principal ventaja de esta estrategia consiste en que elimina la necesidad de utilizar una referencia local de portadora durante la recuperación. Esto simplifica parcialmente la arquitectura de detección y reduce la dependencia sobre mecanismos de sincronización explícita entre modulación y demodulación. Adicionalmente, la detección por envolvente mantiene una relación directa con arquitecturas clásicas de recuperación AM y permite implementar un proceso conceptualmente sencillo.

No obstante, este enfoque también introduce limitaciones importantes dentro del contexto del proyecto. La calidad de recuperación depende considerablemente del comportamiento de la envolvente detectada y del filtrado posterior aplicado sobre la señal. Variaciones sobre amplitud, componentes residuales de alta frecuencia o cambios sobre el índice de modulación pueden afectar la estabilidad de la envolvente estimada y generar distorsión sobre la señal recuperada. En consecuencia, aunque esta alternativa reduce parcialmente la complejidad asociada a sincronización, traslada gran parte del problema hacia el diseño del filtrado y la estabilidad de la detección de amplitud.

La segunda alternativa corresponde a la demodulación coherente. Bajo este enfoque, la señal modulada es multiplicada directamente por una referencia local de portadora equi-

valente a la utilizada durante la modulación:

$$y[n] = s[n] \cdot c[n] \quad (4.7)$$

Sustituyendo la señal modulada en la ecuación (4.7), se obtiene:

$$y[n] = (1 + \mu m[n]) c^2[n] \quad (4.8)$$

Debido a que esta multiplicación genera simultáneamente componentes de baja y alta frecuencia, la señal resultante requiere una etapa posterior de filtrado pasa bajos:

$$\hat{m}[n] = LPF\{y[n]\} \quad (4.9)$$

La principal ventaja de este enfoque corresponde al mayor control matemático que proporciona sobre el proceso de recuperación. Debido a que la detección utiliza una referencia conocida de portadora, la reconstrucción de señal resulta menos dependiente del comportamiento instantáneo de la envolvente y mantiene una recuperación más estable frente a variaciones moderadas de amplitud.

Adicionalmente, dentro de la arquitectura propuesta, la portadora utilizada durante la modulación es generada internamente por el propio sistema embebido. Esto permite reutilizar directamente la referencia necesaria para la detección sin incorporar mecanismos complejos adicionales de sincronización, lo cual favorece la compatibilidad de este enfoque con la estructura general del sistema.

Sin embargo, la demodulación coherente también incrementa la cantidad de operaciones requeridas durante el procesamiento continuo. La necesidad de realizar multiplicaciones adicionales por muestra y mantener la referencia de portadora implica un costo computacional mayor respecto a una detección por envolvente simple. Bajo este escenario, la decisión no depende únicamente del método de recuperación seleccionado, sino también del tipo de filtrado que el sistema puede sostener sin comprometer la estabilidad temporal del procesamiento continuo.

Para la etapa de filtrado posterior se consideran igualmente dos alternativas. La primera consiste en utilizar filtros digitales de mayor selectividad frecuencial, capaces de proporcionar una reconstrucción más precisa de la señal recuperada. De forma general, esta alternativa puede representarse como:

$$\hat{m}[n] = \sum_{k=0}^M b_k y[n - k] \quad (4.10)$$

donde b_k representa los coeficientes del filtro y M su orden.

La ventaja de este enfoque corresponde al mayor control sobre la respuesta frecuencial del sistema y a la posibilidad de mejorar la calidad de reconstrucción de la señal. Sin embargo,

este tipo de filtros incrementa considerablemente la cantidad de multiplicaciones, sumas y almacenamiento temporal requeridos durante el procesamiento continuo, aumentando simultáneamente el consumo de memoria y tiempo de ejecución.

Como alternativa más ligera, la arquitectura propuesta incorpora un filtro pasa bajos recursivo de primer orden:

$$\hat{m}[n] = \hat{m}[n - 1] + \alpha (y[n] - \hat{m}[n - 1]) \quad (4.11)$$

donde α representa el coeficiente de suavizado asociado al filtro.

Aunque esta estructura no proporciona el mismo nivel de selectividad que un filtro digital de mayor orden, reduce considerablemente el costo computacional asociado al procesamiento continuo y únicamente requiere conservar el estado previo de la salida filtrada.

A partir de este análisis, la arquitectura final se orienta hacia una demodulación coherente complementada mediante un filtro pasa bajos recursivo de primer orden. Esta decisión permite evitar la alta dependencia de la detección por envolvente respecto al filtrado y comportamiento de amplitud, sin trasladar simultáneamente al microcontrolador una carga excesiva de procesamiento asociada a filtros digitales más complejos. En consecuencia, la solución seleccionada proporciona un balance adecuado entre estabilidad de recuperación, simplicidad matemática y viabilidad computacional dentro del entorno embebido.

Adaptación de la técnica ASK/OOK

La adaptación de la técnica ASK/OOK introduce un problema distinto al presente en AM, debido a que el sistema requiere generar información digital continua sin depender de almacenamiento extensivo de datos ni comunicación permanente con fuentes externas. Debido a esto, la principal dificultad no consiste únicamente en aplicar la ecuación de modulación digital, sino en definir cómo construir una señal binaria continua compatible con las restricciones computacionales presentes.

Una primera alternativa consiste en utilizar secuencias binarias precargadas dentro de memoria. Esto implica que la generación digital se basa en recorrer patrones previamente almacenados, simplificando parcialmente la arquitectura de generación de señal debido a que el sistema únicamente debe leer estados binarios previamente definidos, donde $b[n] \in \{0,1\}$.

La principal ventaja de esta estrategia corresponde a su simplicidad operacional. Debido a que los estados lógicos ya existen dentro de memoria, el procesamiento requerido para generar la señal digital resulta considerablemente bajo. Esto favorece la estabilidad temporal del sistema y reduce el costo computacional asociado a la generación continua de datos digitales.

Sin embargo, esta alternativa también introduce limitaciones importantes sobre escalabilidad y flexibilidad de la señal generada. Conforme aumenta el tamaño de las secuencias

binarias utilizadas, también aumenta el uso de memoria requerido por el sistema embebido. Adicionalmente, la reutilización continua de patrones estáticos limita considerablemente la variabilidad de la señal y reduce la capacidad del sistema para representar comportamientos digitales más dinámicos. Debido a esto, el sistema prioriza simplicidad y estabilidad operativa, pero sacrifica flexibilidad y capacidad de generación autónoma de información digital.

Como segunda alternativa, se considera la generación pseudoaleatoria controlada mediante un valor de semilla (seed). En este enfoque, la señal digital no es almacenada explícitamente ni transmitida continuamente desde el exterior, sino generada matemáticamente dentro del propio entorno embebido utilizando estructuras ligeras de memoria y operaciones aritméticas simples.

La generación pseudoaleatoria puede representarse mediante una relación recursiva discreta:

$$r[n+1] = (ar[n] + c) \text{ mód } M \quad (4.12)$$

donde $r[n]$ corresponde al estado pseudoaleatorio interno, mientras que a , c y M representan constantes asociadas al generador.

La principal ventaja de esta estrategia corresponde a que permite construir patrones digitales reproducibles utilizando únicamente un estado interno reducido y operaciones matemáticas elementales. Esto elimina la necesidad de almacenar secuencias binarias extensas y mantiene simultáneamente independencia operativa respecto al sistema externo. Adicionalmente, el uso de una semilla permite controlar la generación de patrones y reproducir secuencias idénticas bajo las mismas condiciones de inicialización, lo cual resulta particularmente útil para análisis comparativos y validación de comportamiento.

Sin embargo, la generación pseudoaleatoria no representa directamente estados binarios. Debido a esto, la arquitectura requiere incorporar un mecanismo de decisión que transforme los valores pseudoaleatorios generados en estados lógicos discretos. Para resolver este problema, la señal digital se obtiene mediante una comparación contra un umbral de decisión:

$$b[n] = \begin{cases} 1, & r[n] > \tau \\ 0, & r[n] \leq \tau \end{cases} \quad (4.13)$$

donde $r[n]$ representa el valor pseudoaleatorio generado y τ el umbral de decisión.

Una vez obtenida la secuencia binaria, la modulación ASK/OOK se implementa mediante una relación directa entre el estado lógico y la portadora:

$$s[n] = b[n] \cdot c[n] \quad (4.14)$$

donde $b[n]$ corresponde a la señal binaria discreta y $c[n]$ a la portadora.

La utilización de esta formulación permite mantener una arquitectura matemática considerablemente más simple que otros esquemas de modulación digital más complejos. Debido a que la técnica únicamente requiere activar o desactivar la portadora dependiendo del estado lógico transmitido, el procesamiento asociado a la modulación se reduce principalmente a operaciones de comparación y multiplicación. Esto favorece considerablemente la estabilidad temporal del sistema y disminuye el costo computacional asociado al procesamiento continuo de señal dentro del entorno embebido.

La recuperación de señal digital introduce una segunda decisión crítica asociada al mecanismo de detección utilizado. Una primera alternativa corresponde a mecanismos avanzados de recuperación simbólica y sincronización digital. Bajo este enfoque, la detección incorpora estructuras orientadas a mantener alineamiento temporal continuo entre la señal recibida y el proceso de reconstrucción binaria, incluyendo recuperación de reloj, sincronización de símbolos y técnicas de detección digital más elaboradas.

Conceptualmente, este tipo de arquitecturas puede representarse mediante un proceso general de sincronización simbólica:

$$\hat{b}[k] = D\{s[n], \phi[n], T_b\} \quad (4.15)$$

donde $\phi[n]$ representa parámetros de sincronización temporal y T_b el periodo simbólico asociado a cada bit.

La principal ventaja de esta estrategia corresponde al mayor nivel de robustez frente a ruido, desalineamiento temporal y variaciones sobre el comportamiento de la señal recibida. Debido a que el sistema mantiene control explícito sobre sincronización de símbolos y recuperación temporal, la detección binaria puede conservar mayor precisión bajo condiciones más complejas de transmisión.

Sin embargo, estos mecanismos introducen estructuras considerablemente más complejas para mantener sincronización continua entre la señal recibida y el proceso de detección. La recuperación simbólica requiere procesamiento temporal adicional, almacenamiento de estados internos y control más elaborado sobre el flujo de datos digitales. En consecuencia, aunque esta alternativa proporciona mayor robustez matemática y estabilidad sobre la detección, también incrementa significativamente la carga computacional y la complejidad del sistema, incrementando a su vez el riesgo de fallos en la sincronización y el funcionamiento general.

Como alternativa, la arquitectura propuesta utiliza una estrategia coherente simplificada basada nuevamente en multiplicación directa entre la señal recibida y una portadora local, siguiendo el mismo principio matemático presentado en la ecuación 4.7.

Posteriormente, la señal detectada se procesa mediante el mismo filtro pasa bajos recursivo de primer orden definido en la ecuación 4.11. En este caso, la estructura no se utiliza para reconstruir una señal analógica continua, sino para suavizar la componente detectada antes de aplicar la decisión binaria. Esta reutilización del mismo esquema de filtrado

permite mantener consistencia entre técnicas y reducir la complejidad de implementación dentro del entorno embebido.

Finalmente, la decisión lógica se obtiene mediante comparación contra un umbral:

$$b_d[n] = \begin{cases} 1, & \hat{b}[n] > \tau \\ 0, & \hat{b}[n] \leq \tau \end{cases} \quad (4.16)$$

La principal ventaja de esta estrategia corresponde a que permite mantener una arquitectura considerablemente más ligera desde el punto de vista computacional. Debido a que la detección utiliza únicamente multiplicación directa, filtrado recursivo simple y comparación contra umbral, el sistema evita incorporar mecanismos avanzados de sincronización simbólica que podrían comprometer la estabilidad temporal del procesamiento continuo dentro del microcontrolador.

Adicionalmente, dentro de la arquitectura propuesta, la portadora utilizada durante la modulación es generada internamente por el propio sistema embebido. Esto permite reutilizar directamente la referencia necesaria para la detección coherente sin requerir procesos adicionales complejos de sincronización.

La diferencia entre ambas alternativas no corresponde a que una resulte técnicamente superior de forma absoluta, sino al balance entre robustez matemática y viabilidad computacional que cada una representa. Los mecanismos avanzados de sincronización digital ofrecen mayor precisión y tolerancia frente a desalineamiento temporal, pero incrementan considerablemente la complejidad del procesamiento embebido. Por otro lado, la estrategia coherente simplificada reduce parte de esa robustez adicional, pero mantiene una arquitectura considerablemente más estable y compatible con las restricciones reales del sistema.

Integración de controles analógicos

A partir de la estructura física presentada en la Figura 4.2, los periféricos analógicos del sistema se reutilizan como una interfaz de control común para las distintas técnicas implementadas. Por esta razón, los potenciómetros identificados como pot1, pot2 y pot3 adquieren una funcionalidad distinta dependiendo de la técnica activa dentro del sistema.

La Tabla 4.3 presenta la asignación funcional utilizada para cada técnica implementada.

Tabla 4.3: Asignación de controles analógicos por técnica implementada.

Control	AM	ASK/OOK
pot1	Frecuencia de mensaje	Valor de semilla
pot2	Frecuencia de portadora	Frecuencia de portadora
pot3	Índice de modulación μ	Umbral de decisión τ

La reutilización de la misma estructura física entre técnicas reduce complejidad de hardware y evita incorporar interfaces de control independientes para cada algoritmo implementado. Aunque cada parámetro adquiere un significado funcional distinto dependiendo de la técnica activa, la arquitectura de adquisición analógica permanece desacoplada de la lógica específica de modulación y demodulación.

Manejo del flujo de datos

La ejecución continua del sistema requiere mantener un flujo organizado de datos entre las distintas etapas internas de procesamiento, desde la generación de la señal hasta su posterior transmisión hacia la aplicación de monitoreo. En este contexto, la arquitectura debe permitir que la señal original, la señal modulada y la señal demodulada permanezcan temporalmente disponibles dentro del entorno embebido para conservar sincronización entre todas las etapas del proceso.

Para resolver este problema, la arquitectura se organiza mediante procesamiento por bloques, donde cada conjunto de muestras representa una unidad funcional completa de señal. Inicialmente se genera un bloque correspondiente a la señal original, el cual posteriormente es utilizado como entrada para la etapa de modulación. A partir de este bloque se construye una segunda representación asociada a la señal modulada y, seguidamente, una tercera representación correspondiente a la señal demodulada.

En lugar de sobrescribir continuamente cada etapa de procesamiento, la arquitectura conserva simultáneamente las tres versiones de la señal dentro de buffers independientes. Esto permite mantener alineación temporal entre las distintas representaciones y evita pérdidas de coherencia entre el proceso de generación, modulación y recuperación de señal.

Una vez completado el procesamiento interno del bloque, las tres etapas de señal son agrupadas dentro de un mismo frame serial estructurado para posteriormente ser transmitidas hacia el sistema de monitoreo.

4.2.2. Desarrollo del sistema de visualización y monitoreo

La implementación del sistema embebido requiere no solamente ejecutar los procesos de modulación y demodulación sobre el RP2040, sino también disponer de un mecanismo que permita observar y analizar continuamente el comportamiento de las señales procesadas durante la ejecución del sistema. Por lo tanto, la visualización pasa a convertirse en un componente fundamental para verificar el funcionamiento de las técnicas implementadas y analizar los efectos producidos sobre la señal durante cada etapa del procesamiento.

A diferencia de un entorno de simulación, en donde el procesamiento y la visualización se ejecutaban dentro de un mismo entorno local, en este proyecto el procesamiento principal ocurre sobre un sistema embebido y la visualización se realiza en un entorno externo.

Esto introduce la necesidad de transmitir continuamente información fuera del sistema embebido, donde posteriormente las señales van a ser reconstruidas y visualizadas.

Debido a esto, el sistema de monitoreo se plantea como una plataforma orientada a visualizar simultáneamente la señal original, la señal modulada y la señal demodulada, permitiendo comparar continuamente el comportamiento de cada una durante la ejecución del sistema. Esta necesidad introduce restricciones asociadas al manejo continuo de datos, la estabilidad visual, la latencia de actualización y la sincronización temporal entre bloques de información transmitidos mediante comunicación serial.

Diseño del módulo de visualización

Conociendo que el sistema de monitoreo debe sostener visualización continua de múltiples señales durante la ejecución del sistema, el siguiente paso consiste en definir la arquitectura y las herramientas necesarias para construir una aplicación capaz de lograr este objetivo.

Uno de los aspectos importantes dentro de esta etapa consiste en definir el entorno de desarrollo utilizado para construir la aplicación de monitoreo. Considerando que el entorno embebido se encuentra basado sobre MicroPython, se prioriza mantener Python también como lenguaje principal para la aplicación. Esto permite conservar uniformidad tecnológica entre ambas partes del sistema, reduciendo simultáneamente complejidad asociada a la integración entre distintos lenguajes, manejo de estructuras heterogéneas y adaptación entre diferentes entornos de ejecución. Adicionalmente, Python proporciona un ecosistema particularmente conveniente para aplicaciones orientadas a procesamiento de señales y visualización científica, principalmente debido a la disponibilidad de librerías para comunicación serial, manejo eficiente de estructuras numéricas y construcción de interfaces gráficas.

Teniendo a Python como entorno principal, el siguiente paso consiste en seleccionar el framework gráfico y las herramientas de visualización capaces de sostener actualización continua de señales sin degradar significativamente el rendimiento del sistema. Debido a esto, se selecciona PyQt5 como framework principal para el desarrollo de la interfaz de monitoreo, principalmente por su integración con librerías orientadas a renderizado dinámico de información y su capacidad para construir aplicaciones gráficas organizadas alrededor de múltiples regiones funcionales.

A partir de esta arquitectura basada en PyQt5, inicialmente se considera reutilizar parte del enfoque de visualización utilizado en el proyecto antecesor a este, donde la representación gráfica se encontraba basada principalmente en Matplotlib. Sin embargo, aunque Matplotlib constituye una herramienta ampliamente utilizada para representación científica y análisis de datos, comienzan a observarse limitaciones importantes cuando el entorno de ejecución pasa de un escenario de simulación convencional hacia un sistema basado en adquisición continua de información externa.

A diferencia de un enfoque estático, donde las gráficas son actualizadas ocasionalmente o

posterior al procesamiento, el sistema desarrollado requiere refresco continuo de múltiples señales mientras simultáneamente se reciben nuevos bloques de información mediante comunicación serial. Bajo este escenario, el problema deja de estar asociado únicamente a representar señales y pasa a involucrar restricciones relacionadas con frecuencia de actualización, estabilidad temporal, latencia de renderizado y manejo eficiente de buffers gráficos. Debido a esto, en la Tabla 4.4 se realiza una comparación entre Matplotlib y PyQtGraph considerando los criterios mencionados.

Tabla 4.4: Comparación entre Matplotlib y PyQtGraph para visualización continua

Criterio	Matplotlib	PyQtGraph
Enfoque principal	Visualización científica y análisis estático	Visualización dinámica y adquisición en tiempo real
Frecuencia de actualización	Moderada	Alta
Rendimiento en streaming continuo	Limitado	Optimizado para actualización continua
Latencia de renderizado	Mayor	Reducida
Manejo de múltiples curvas dinámicas	Menor eficiencia	Actualización optimizada
Integración con Qt	Parcial	Nativa
Estabilidad visual en ejecución continua	Variable	Más estable
Manejo eficiente de buffers gráficos	Limitado	Optimizado

A partir de los criterios presentados en la Tabla 4.4, se selecciona PyQtGraph como librería principal para el sistema de monitoreo, principalmente por su buen rendimiento en la representación continua de datos. Adicionalmente, PyQtGraph presenta integración directa con Qt mediante estructuras optimizadas para actualización dinámica de curvas, permitiendo reducir considerablemente el costo asociado al refresco continuo de múltiples regiones de visualización.

Asimismo, la interfaz incorpora regiones destinadas al registro continuo de información asociada a la ejecución del sistema y al monitoreo de parámetros relevantes durante la transmisión y procesamiento de señales. La separación de estas regiones respecto al área principal de visualización permite desacoplar la representación gráfica del manejo interno de información de control y supervisión.

Aunque la selección del stack gráfico y la organización general de la interfaz permiten establecer una base estable para el monitor, todavía resulta necesario definir estrategias de transmisión, sincronización y reconstrucción de datos, necesarias para mantener alineación temporal entre las distintas etapas de la señal.

Estructuración del flujo de adquisición y reconstrucción de señales

La integración entre el sistema embebido y la aplicación de monitoreo introduce un problema asociado a cómo transmitir continuamente las distintas etapas del procesamiento sin perder sincronización temporal entre señales durante la adquisición y reconstrucción de información. Debido a que el sistema requiere visualizar simultáneamente la señal original, la señal modulada y la señal demodulada, resulta necesario definir una estrategia que permita mantener asociadas todas las representaciones correspondientes a un mismo instante temporal.

Tomando como referencia la estrategia de procesamiento por bloques descrita previamente en la Sección 4.2.1, el sistema embebido conserva simultáneamente buffers independientes correspondientes a cada etapa de señal. A partir de esta organización interna, una primera alternativa consiste en transmitir cada bloque de manera individual hacia la aplicación de monitoreo, enviando secuencialmente los bloques asociados a un mismo instante temporal de la señal.

La principal ventaja de esta estrategia corresponde a la simplicidad estructural del protocolo de transmisión, debido a que cada bloque puede enviarse individualmente sin requerir mecanismos adicionales de agrupamiento o empaquetado. Esto reduce parcialmente la complejidad computacional asociada al proceso de transmisión dentro del entorno embebido. Sin embargo, este enfoque también traslada gran parte del problema hacia la aplicación de monitoreo, debido a que el sistema externo debe reorganizar continuamente los distintos paquetes recibidos y reconstruir externamente la relación temporal entre señales. Bajo transmisión continua, pequeñas variaciones temporales entre paquetes pueden provocar desalineamiento entre etapas y dificultar simultáneamente la comparación visual directa entre la señal original, la señal modulada y la señal demodulada.

Como alternativa, se plantea una arquitectura basada en frames seriales sincronizados, donde toda la información correspondiente a un mismo bloque temporal permanece agrupada durante el proceso de transmisión. En este contexto, cada frame incorpora simultáneamente la señal original, la señal modulada y la señal demodulada correspondientes al mismo conjunto de muestras, además de información de control asociada a la técnica activa, tamaño del bloque de muestras, frecuencia de muestreo y parámetros actualmente utilizados durante la ejecución del sistema embebido. Posteriormente, toda esta información es transmitida como una única unidad estructurada hacia la aplicación de monitoreo tal y como se muestra en la Figura 4.3.

SYNC	HEADER técnica, N , f_s , parámetros	Payload señal original	Payload señal modulada	Payload señal demodulada
------	---	---------------------------	---------------------------	-----------------------------

Figura 4.3: Estructura general propuesta para el frame serial sincronizado.

La principal ventaja de esta arquitectura corresponde a que permite conservar alineación temporal entre señales durante todo el proceso de adquisición y reconstrucción. Debido a

que todas las representaciones pertenecientes a un mismo instante temporal son transmitidas dentro del mismo frame, la aplicación de monitoreo puede reconstruir directamente la relación entre etapas sin incorporar mecanismos adicionales de sincronización externa.

Considerando estas ventajas, se adopta el enfoque basado en frames seriales sincronizados como arquitectura principal para el sistema de adquisición y visualización. Aunque este enfoque introduce una mayor carga asociada al empaquetado y transmisión de información dentro del entorno embebido, se considera preferible concentrar parcialmente esta complejidad sobre el RP2040 con tal de preservar sincronización temporal entre señales y simplificar considerablemente el proceso de reconstrucción y monitoreo dentro de la aplicación externa.

Una vez establecida la estructura principal de transmisión, aparece un segundo problema asociado a la conmutación dinámica entre técnicas de modulación. Debido a que AM y ASK/OOK utilizan parámetros distintos y producen comportamientos diferentes sobre la interfaz de monitoreo, resulta necesario definir un mecanismo que permita identificar automáticamente cuándo ocurre un cambio de técnica dentro del sistema embebido.

Una primera alternativa consiste en incorporar un mecanismo manual de selección dentro de la propia interfaz gráfica, permitiendo que el usuario indique explícitamente cuál técnica se encuentra activa en el entorno embebido. Bajo este enfoque, el monitor reorganiza internamente sus parámetros y estructuras visuales dependiendo del estado seleccionado por el usuario, permitiendo adaptar la interfaz para representar los parámetros específicos asociados a cada técnica. La principal ventaja de esta estrategia corresponde a su simplicidad de implementación, debido a que la lógica de adquisición serial permanece prácticamente independiente del comportamiento de la interfaz. Adicionalmente, el monitor puede reorganizar directamente sus elementos visuales sin requerir información adicional proveniente del entorno embebido.

No obstante, este enfoque también introduce una dependencia directa entre el estado real del sistema embebido y la interacción manual del usuario dentro de la aplicación de monitoreo. Bajo estas condiciones, inconsistencias entre la técnica verdaderamente activa y la configuración seleccionada en la interfaz pueden provocar errores de interpretación sobre los parámetros recibidos y desalineamiento entre adquisición y visualización.

Como alternativa, se propone una arquitectura desacoplada basada en transmisión dinámica de metadata. Bajo este enfoque, el sistema embebido transmite automáticamente un paquete adicional de control cada vez que ocurre una conmutación entre técnicas. Este paquete incorpora información asociada a la técnica activa, parámetros disponibles y configuraciones requeridas para reorganizar dinámicamente el comportamiento interno del monitor.

A diferencia del frame principal de señal, cuyo objetivo consiste en transmitir muestras sincronizadas para visualización continua, el paquete de metadata se orienta a describir la configuración operativa actual del sistema. Debido a esto, el monitor puede actualizar automáticamente parámetros internos y adaptar dinámicamente su estructura cuando

ocurre una conmutación entre técnicas.

La principal ventaja de esta arquitectura corresponde a que desacopla el comportamiento del monitor respecto a interacción manual del usuario y permite que la aplicación adapte automáticamente su configuración dependiendo de la técnica actualmente ejecutada. En consecuencia, la lógica de adquisición y visualización pasa a depender directamente de la información transmitida desde el entorno embebido, favoreciendo simultáneamente modularidad, sincronización y escalabilidad dentro de la arquitectura general del sistema.

4.2.3. Optimización del procesamiento y flujo de datos en el sistema embebido

La disposición de una arquitectura funcional para la ejecución continua de las técnicas de modulación y demodulación no garantiza por sí sola la estabilidad operativa del sistema embebido. Una vez establecidas las adaptaciones algorítmicas descritas en la Sección 4.2, el análisis del comportamiento real del sistema evidenció que distintos aspectos relacionados con la organización interna del procesamiento, la estructura del protocolo de transmisión serial y la parametrización del sistema podían ser refinados para mejorar el rendimiento global bajo condiciones de operación continua. En consecuencia, el proceso de optimización se orienta hacia dos dimensiones complementarias: el refinamiento del flujo de procesamiento y comunicación serial, y la mejora de la estabilidad temporal junto con la modularidad estructural del sistema.

Refinamiento del flujo de procesamiento y comunicación serial

La arquitectura de procesamiento por bloques establecida en la Sección 4.2.1 permite organizar el flujo interno de datos en tres etapas sucesivas: generación, modulación y demodulación, preservando simultáneamente las tres representaciones de señal dentro de buffers independientes. Sin embargo, el análisis del comportamiento operativo del sistema identificó que la organización inicial de este flujo y la estructura del frame serial podían optimizarse para reducir ineficiencias asociadas a la asignación de estructuras temporales, la secuencia de empaquetado de datos y el mecanismo de sincronización utilizado en la recepción.

Un primer aspecto de refinamiento corresponde al manejo de los buffers internos de señal. En el primer enfoque de implementación, cada etapa del procesamiento generaba estructuras de datos intermedias de forma independiente, sin aprovechar la posibilidad de reutilizar espacios de memoria previamente reservados. Bajo las restricciones de la plataforma embebida, esta práctica incrementa innecesariamente la presión sobre la memoria SRAM disponible y aumenta la frecuencia de operaciones de asignación y liberación de memoria dentro del ciclo de procesamiento continuo. La alternativa adoptada consiste en reservar de forma estática los buffers asociados a las tres etapas de señal desde el inicio del programa y reutilizarlos en cada iteración de procesamiento sin reinicialización inne-

cesaria de estructuras. Esta decisión reduce el costo asociado a la gestión dinámica de memoria y contribuye a mantener un comportamiento temporal más predecible durante la ejecución continua.

Un segundo aspecto de refinamiento afecta directamente a la estructura del protocolo de comunicación serial hacia la aplicación de monitoreo. El diseño del protocolo establece una separación explícita entre dos tipos de información transmitida: los metadatos de configuración del sistema y los frames de señal procesada. Los metadatos incluyen la identificación de la técnica activa, la asignación funcional de los potenciómetros y los parámetros generales del sistema, mientras que los frames de señal transportan las tres etapas de procesamiento en formato binario compacto. Ambos tipos de mensaje utilizan secuencias de sincronización independientes, lo que permite que la aplicación receptora los distinga e interprete de forma diferenciada sin ambigüedad.

Una decisión clave dentro de este protocolo corresponde a la transmisión condicional de metadatos. Dado que la configuración del sistema permanece estable mientras la técnica activa no cambia, el envío de metadatos se realiza únicamente cuando se detecta un cambio en la técnica seleccionada mediante el interruptor físico. Esta estrategia evita saturar el canal serial con información redundante en cada ciclo de procesamiento y reserva el ancho de banda disponible exclusivamente para la transmisión continua de frames de señal durante la operación normal del sistema.

El frame de señal se construye como una estructura binaria compacta que concatena un encabezado de control con los tres bloques de muestras correspondientes a la señal original, modulada y demodulada:

$$\text{Frame} = \text{HDR} \parallel \text{payload}_m \parallel \text{payload}_{mod} \parallel \text{payload}_{dem} \quad (4.17)$$

donde $\parallel$ denota concatenación secuencial de bloques binarios. El encabezado incluye el identificador de técnica activa, la cantidad de muestras por bloque, la frecuencia de muestreo y los valores actuales de los tres potenciómetros. La codificación binaria de cada campo utiliza tipos de datos de longitud fija seleccionados según la naturaleza del valor representado, lo que reduce el tamaño total del frame transmitido en comparación con alternativas basadas en texto o punto flotante de mayor precisión.

La decisión de agrupar las tres etapas de señal dentro de un mismo frame responde a una restricción funcional de la aplicación de monitoreo. Dado que el sistema visualiza simultáneamente la señal original, la modulada y la demodulada en tiempo real, un esquema que transmita cada etapa de forma independiente introduciría desalineamiento temporal entre los bloques graficados. La transmisión unificada garantiza que los tres bloques correspondan siempre al mismo intervalo temporal de procesamiento, permitiendo que la aplicación los grafique de forma sincronizada sin mecanismos adicionales de alineación en recepción.

La Tabla 4.5 resume la estructura del frame serial utilizado por el sistema.

Tabla 4.5: Estructura del frame serial transmitido por el sistema embebido.

Campo	Descripción	Tamaño
Sincronización de datos	Secuencia fija de inicio de frame de señal	4 bytes
Identificador de técnica	Técnica activa en el ciclo actual	1 byte
Cantidad de muestras	Valor de N por bloque	2 bytes
Frecuencia de muestreo	Valor de F_s del sistema	2 bytes
Valores de potenciómetros	Estado actual de pot1, pot2, pot3	12 bytes
Señal original	N muestras en formato binario compacto	$2N$ bytes
Señal modulada	N muestras en formato binario compacto	$2N$ bytes
Señal demodulada	N muestras en formato binario compacto	$2N$ bytes

Optimización de estabilidad temporal y modularidad

Además del refinamiento del protocolo de comunicación, el análisis del sistema operando bajo condiciones continuas identificó factores adicionales que condicionan la estabilidad temporal del procesamiento. Dos de ellos resultan especialmente determinantes: el tamaño del bloque de procesamiento N y las limitaciones temporales inherentes a la ejecución de MicroPython sobre el microcontrolador RP2040.

La selección del tamaño de bloque N representa una decisión de ingeniería con implicaciones directas sobre tres variables interdependientes: la latencia de visualización, la carga computacional por ciclo de procesamiento y la frecuencia de actualización de la interfaz gráfica (FPS). Un bloque de mayor tamaño reduce la frecuencia de iteraciones del ciclo principal y, en consecuencia, disminuye la sobrecarga asociada a la inicialización repetida de estructuras y operaciones de empaquetado serial. Sin embargo, incrementa simultáneamente el tiempo requerido para completar cada ciclo de procesamiento, lo que introduce mayor latencia entre la modificación de un parámetro físico —mediante potenciómetro— y su reflejo visual en la aplicación de monitoreo. Por otro lado, un bloque excesivamente pequeño reduce la latencia pero incrementa la frecuencia de operaciones de transmisión serial y empaquetado, lo que puede comprometer la estabilidad temporal del sistema bajo las limitaciones del intérprete de MicroPython.

MicroPython introduce una restricción fundamental sobre la ejecución de operaciones matemáticas iterativas dentro de ciclos de procesamiento continuo. A diferencia de implementaciones compiladas en C o C++, la interpretación dinámica del código Python genera una sobrecarga de ejecución no despreciable por cada iteración del ciclo de proce-

samiento de muestras. Esta sobrecarga se acumula a medida que aumenta el número de muestras procesadas por bloque, limitando la frecuencia máxima de actualización sostenible sin comprometer la estabilidad temporal del sistema. En consecuencia, el balance entre tamaño de bloque, latencia y FPS no puede determinarse exclusivamente desde consideraciones matemáticas teóricas, sino que requiere validación empírica bajo las condiciones reales de ejecución sobre el hardware seleccionado.

A partir de este análisis, la arquitectura del sistema define el tamaño de bloque como un parámetro configurable externamente, en lugar de fijarlo como constante dentro del código del microcontrolador. Esta decisión permite ajustar el comportamiento temporal del sistema en función de las condiciones de operación sin requerir modificaciones sobre la lógica de procesamiento interna.

La segunda dimensión de optimización corresponde a la modularidad estructural del sistema embebido. La implementación inicial organiza el procesamiento dentro de un archivo principal `main.py`, complementado por archivos independientes para cada técnica implementada (`AM.py` y `ASK.py`). Aunque esta organización resulta funcional, no establece una interfaz genérica que permita incorporar técnicas adicionales de forma sistemática. La ausencia de una estructura común obliga a realizar modificaciones sobre el archivo principal cada vez que se integra una nueva técnica, incrementando el riesgo de introducir errores de integración y dificultando el mantenimiento del sistema a medida que aumenta la cantidad de técnicas soportadas.

El refinamiento adoptado establece una arquitectura modular basada en una interfaz genérica para los archivos de técnica. Bajo este esquema, cada técnica implementa un conjunto de funciones con nombres y firmas estandarizados, de forma que el archivo principal puede invocar las operaciones de modulación y demodulación sin conocer los detalles internos de implementación de cada técnica. Esta decisión permite incorporar futuras técnicas de modulación con únicamente desarrollar el archivo correspondiente bajo el formato establecido, sin modificar la lógica central del sistema. La Tabla 4.6 resume la estructura genérica adoptada para los archivos de técnica.

Tabla 4.6: Interfaz genérica establecida para los módulos de técnica en el sistema embebido.

Función	Responsabilidad
<code>generate_signal(params)</code>	Generación de la señal de información según parámetros del potenciómetro
<code>modulate(signal, params)</code>	Aplicación del algoritmo de modulación correspondiente a la técnica
<code>demodulate(signal, params)</code>	Recuperación de la señal de información mediante el algoritmo de demodulación

La tercera dimensión de optimización aborda la parametrización del sistema mediante

un archivo de configuración externo en formato JSON (`config.json`). La motivación principal de esta decisión deriva de la necesidad de separar los parámetros de operación del sistema de la lógica de procesamiento implementada en el código. En la implementación inicial, parámetros como la frecuencia de muestreo F_s , el tamaño de bloque N y los rangos de mapeo de los potenciómetros para cada técnica se encuentran definidos directamente dentro del código del microcontrolador, lo que implica que cualquier ajuste sobre estos valores requiere modificar y recargar el firmware del dispositivo.

La incorporación del archivo `config.json` resuelve este problema al centralizar en un único archivo todos los parámetros configurables del sistema. El archivo define dos categorías de parámetros: los parámetros generales del sistema —frecuencia de muestreo, tamaño de bloque, velocidad de transmisión serial— y los parámetros específicos de cada técnica, que incluyen los rangos de mapeo asociados a cada potenciómetro. La estructura del archivo permite que la aplicación de monitoreo en PC incluya una herramienta interactiva de configuración que permite al usuario modificar estos parámetros de forma visual y regenerar el archivo de configuración sin intervenir directamente sobre el código del microcontrolador.

Desde la perspectiva del sistema embebido, el archivo `config.json` es leído durante la inicialización del programa y sus valores se utilizan para configurar dinámicamente los rangos de operación de cada potenciómetro según la técnica activa. Esto permite que un mismo hardware físico opere con distintos rangos paramétricos según la técnica seleccionada, sin requerir reconfiguraciones manuales del firmware. La Tabla 4.7 presenta los parámetros incluidos en el archivo de configuración del sistema.

Tabla 4.7: Parámetros definidos en el archivo de configuración `config.json`.

Categoría	Parámetro	Descripción
General	<code>fs</code>	Frecuencia de muestreo del sistema
	<code>n_samples</code>	Tamaño del bloque de procesamiento N
	<code>baud_rate</code>	Velocidad de transmisión serial
Por técnica	<code>pot1_range</code>	Rango de mapeo del potenciómetro 1
	<code>pot2_range</code>	Rango de mapeo del potenciómetro 2
	<code>pot3_range</code>	Rango de mapeo del potenciómetro 3

En conjunto, las optimizaciones descritas en esta subsección responden a un conjunto de restricciones técnicas identificadas durante la operación real del sistema. La reutilización

de buffers y la codificación compacta de muestras reducen la presión sobre la memoria disponible y el ancho de banda serial. El refinamiento del protocolo de sincronización fortalece la robustez de la comunicación frente a condiciones de desalineamiento temporal. La parametrización del tamaño de bloque proporciona un mecanismo de ajuste adaptable frente a las limitaciones temporales de MicroPython. La modularización por técnicas facilita la extensibilidad del sistema sin comprometer la estabilidad del núcleo de procesamiento. Y la incorporación del archivo de configuración desacopla los parámetros operativos del código, favoreciendo una administración más flexible y mantenible del sistema embebido en su conjunto.

4.2.4. Implementación del proceso de evaluación y análisis de desempeño

Selección e integración de métricas de evaluación

Evaluación de estabilidad y comportamiento temporal

4.3. Análisis de los resultados obtenidos

4.3.1. Resultados asociados a la integración de los algoritmos de modulación y demodulación

4.3.2. Resultados asociados al sistema de visualización y análisis de señales

4.3.3. Resultados asociados a la optimización del procesamiento y flujo de datos

4.3.4. Resultados asociados al proceso de evaluación y análisis de desempeño

Capítulo 5

Conclusiones y Recomendaciones

Capítulo 6

Apéndices y Anexos

Bibliografía

- [1] S. Haykin, *Communication Systems*, 4th. New York, USA: Wiley, 2001.
- [2] A. V. Oppenheim y A. S. Willsky, *Signals and Systems*, 2nd. Upper Saddle River, NJ, USA: Prentice Hall, 1997.
- [3] S. Heath, *Embedded Systems Design*. Oxford, UK: Newnes, 2002.
- [4] Tecnológico de Costa Rica. «Instituto Tecnológico de Costa Rica.» Accessed: Apr. 2026. Available at: <https://www.tec.ac.cr>.
- [5] Y. Su, L. Dong, Z. Zhou, X. Liu y X. Wei, «A General Embedded Underwater Acoustic Communication System Based on Advanced STM32,» *IEEE Embedded Systems Letters*, vol. 13, n.º 3, págs. 90-93, 2021. DOI: [10.1109/LES.2020.3006838](https://doi.org/10.1109/LES.2020.3006838)
- [6] J. Yan, L. Cui, X. Yang, C. Chen y X. Guan, «Design of an Embedded System for Integrated Underwater Communication and Detection,» *IEEE Embedded Systems Letters*, vol. 17, n.º 2, págs. 83-86, 2025. DOI: [10.1109/LES.2024.3485608](https://doi.org/10.1109/LES.2024.3485608)
- [7] Y. Xie, J. Wang, L. Lei, A. Tang, W. Tang y W. Lai, «Design and Implementation of AM Signal Modulation and Demodulation System Based on STM32 and ZYNQ,» en *Proc. 4th Int. Conf. Electronic Information Engineering and Computer Communication (EIECC)*, 2024, págs. 280-285. DOI: [10.1109/EIECC64539.2024.10929487](https://doi.org/10.1109/EIECC64539.2024.10929487)
- [8] J. I. M.-A. et al., «A Power-Line Communication System Governed by Loop Resonance for Photovoltaic Plant Monitoring,» *Sensors*, vol. 22, n.º 23, pág. 9207, 2022. DOI: [10.3390/s22239207](https://doi.org/10.3390/s22239207)
- [9] B. P. Lathi y Z. Ding, *Modern Digital and Analog Communication Systems*, 4th. Oxford, UK: Oxford University Press, 2009.
- [10] J. G. Proakis y M. Salehi, *Digital Communications*, 5th. New York, USA: McGraw-Hill, 2007.
- [11] C. E. Shannon, «Communication in the Presence of Noise,» *Proceedings of the IRE*, vol. 37, n.º 1, págs. 10-21, 1949.
- [12] B. Sklar, *Digital Communications: Fundamentals and Applications*, 2nd. Upper Saddle River, NJ, USA: Prentice Hall, 2001.
- [13] S. K. Mitra, *Digital Signal Processing: A Computer-Based Approach*, 3rd. New York, USA: McGraw-Hill, 2006.

- [14] M. Wolf, *Computers as Components: Principles of Embedded Computing System Design*, 3rd. Burlington, MA, USA: Morgan Kaufmann, 2012.
- [15] T. Noergaard, *Embedded Systems Architecture: A Comprehensive Guide for Engineers and Programmers*. Oxford, UK: Elsevier, 2005.
- [16] S. W. Smith, *The Scientist and Engineer's Guide to Digital Signal Processing*. San Diego, CA, USA: California Technical Publishing, 1997.
- [17] H. Kopetz, *Real-Time Systems: Design Principles for Distributed Embedded Applications*, 2nd. New York, USA: Springer, 2011.
- [18] R. S. Pressman, *Software Engineering: A Practitioner's Approach*, 7th. New York, USA: McGraw-Hill, 2010.
- [19] Raspberry Pi Ltd, *Raspberry Pi Pico Documentation*, Accessed: 2026-05-16, 2026. Available at: <https://www.raspberrypi.com/documentation/microcontrollers/raspberry-pi-pico.html>.